Inteligencia Artificial

Agentes y resolución de problemas mediante búsqueda

Ing. González H. Alexis



© gonzalezhalexis@gmail.com ©

Tabla de Contenidos

01

Active Recall

Repaso del tema anterior

02

Main Content

Historia de la IA
Agentes
Resolución de problemas mediante búsqueda

03

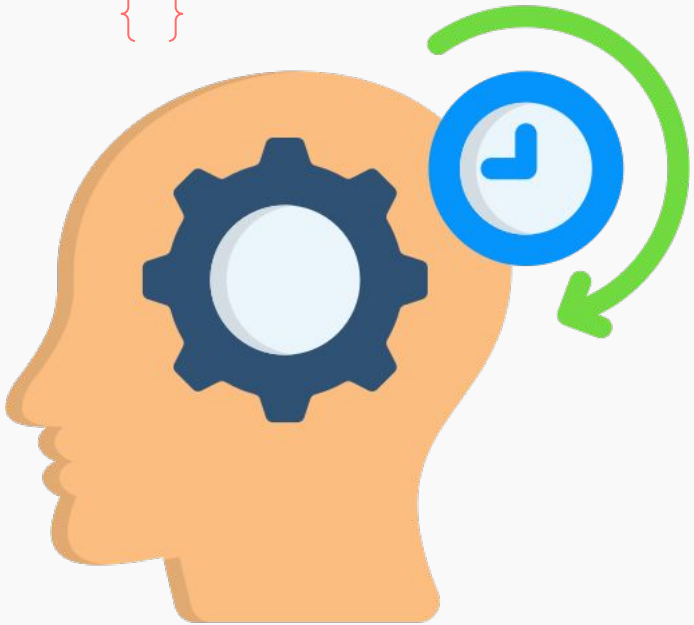
Live Coding Pair Programming

Ejercitación & Feedback

© gonzalezhalexis@gmail.com ©

01

Active Recall



© gonzalezhalexis@gmail.com ©

Inteligencia Artificial

¿Qué es la IA?

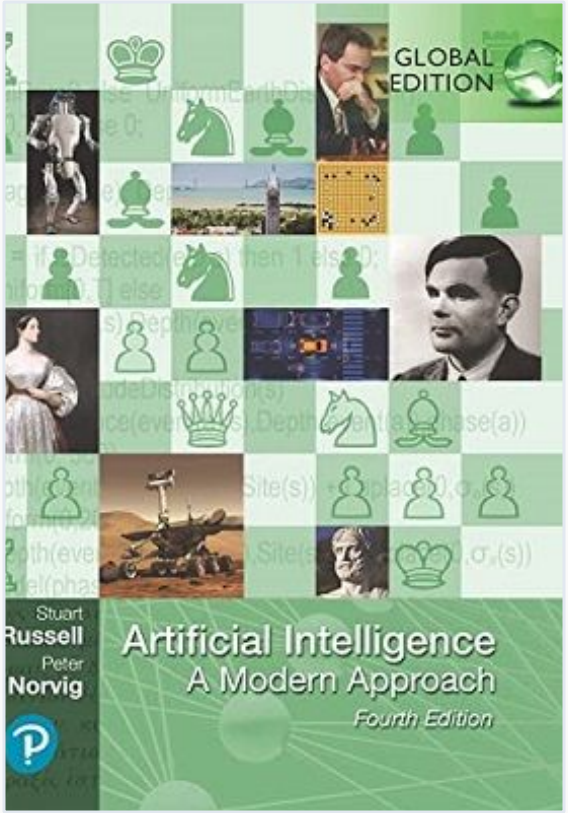
Como en todo campo de vanguardia, no existe una sola definición. Según **Stuart Russell y Peter Norvig**, se analiza desde dos enfoques principales:

A veces se define en función de:

- **Fidelidad humana:** La exactitud del desempeño humano (u otro animal).
- **Racionalidad:** Hacer "lo correcto" de manera ideal u óptima.

También se considera una propiedad:

- **Mente e Inteligencia:** De los procesos de pensamiento y razonamiento internos.
- **Actuación:** Del comportamiento, es decir, una característica externa observable.



© gonzalezhalexis@gmail.com ©

Inteligencia Artificial

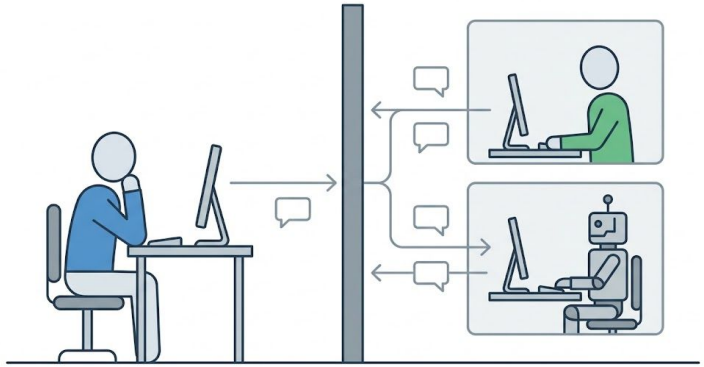
¿Qué es el Test de Turing?

Propuesto por Alan Turing (1950), evalúa si la conducta de una máquina es indistinguible de la de un ser humano en una prueba a ciegas.

Elementos y Funcionamiento

- **Interrogador (Humano):** Formula preguntas por texto sin ver a los participantes.
- **Muro / Aislamiento:** Elimina cualquier señal física o sensorial.
- **Sujetos (Humano + IA):** Responden desde el otro lado intentando convencer al evaluador.
- **Criterio de Éxito:** Si la IA engaña al evaluador, se considera que actúa humanamente.

Esquema del Experimento



Evaluador | Humano | IA / Computadora

El evaluador analiza únicamente las respuestas textuales para determinar la naturaleza de cada sujeto.

© gonzalezhalexis@gmail.com ©

Inteligencia Artificial

Actuando humanamente: El Test de Turing

Programar software para pasar el test requiere desarrollos en áreas clave de la IA:

Procesamiento del Lenguaje Natural

Comunicación exitosa en lenguaje humano.

Representación del Conocimiento

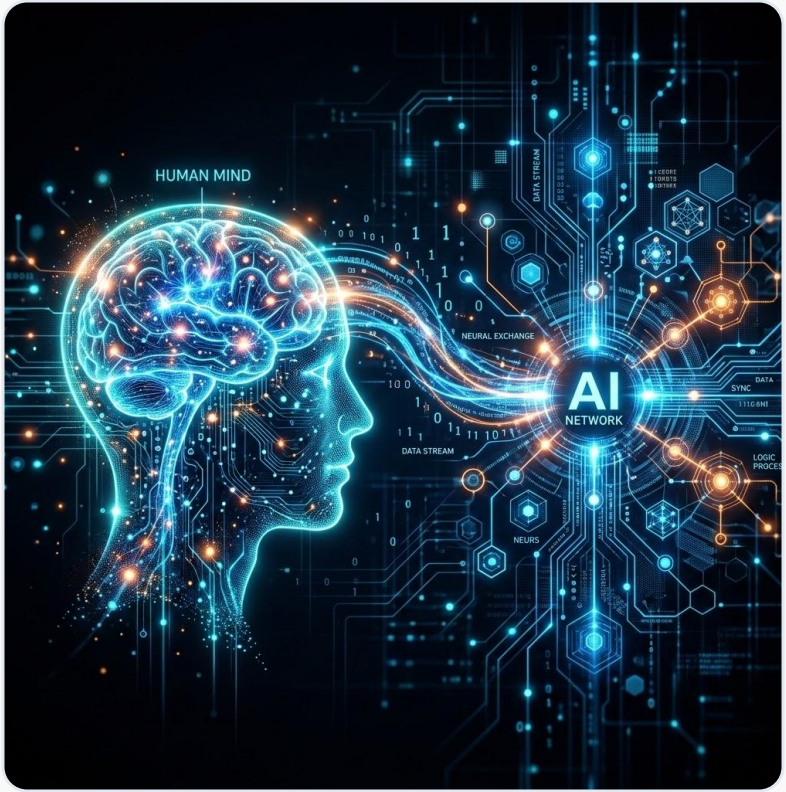
Almacenamiento de información conocida o recibida.

Razonamiento Automático

Responder preguntas y extraer nuevas conclusiones.

Aprendizaje Automático

Adaptación a nuevas circunstancias y extrapolación de patrones.



© gonzalezhalexis@gmail.com ©

Inteligencia Artificial

Pensando Racionalmente

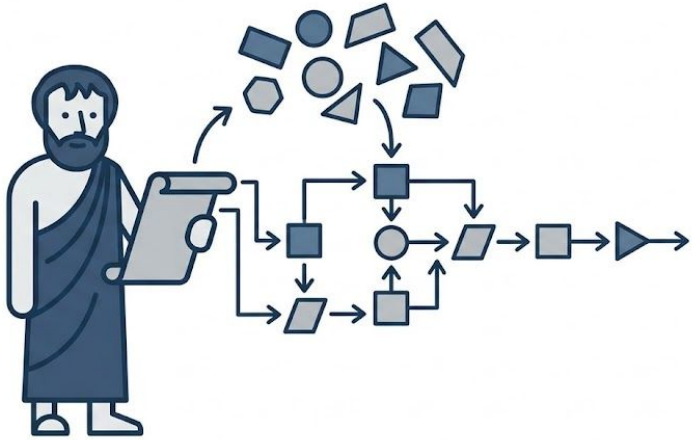
El filósofo Aristóteles fue el primero en intentar codificar el "pensar correctamente", sentando las bases de la lógica formal.

Lógica y Automatización

Silogismos: Patrones de argumentación que garantizan conclusiones correctas si las premisas son válidas.

- *"Sócrates es un hombre y todos los hombres son mortales, entonces Sócrates es mortal"*
- **Notación Formal (S. XVIII):** Lenguaje preciso para definir objetos y sus relaciones.
- **Resolución por Software (1965):** Programas capaces de resolver computacionalmente cualquier problema expresado en notación lógica.

Estructura del Razonamiento Lógico



© gonzalezhalexis@gmail.com ©

Inteligencia Artificial

Incertidumbre y Probabilidad

La lógica espera que el conocimiento del mundo sea cierto, pero está regido por la incertidumbre. La teoría de probabilidad permite un razonamiento riguroso con información incierta.

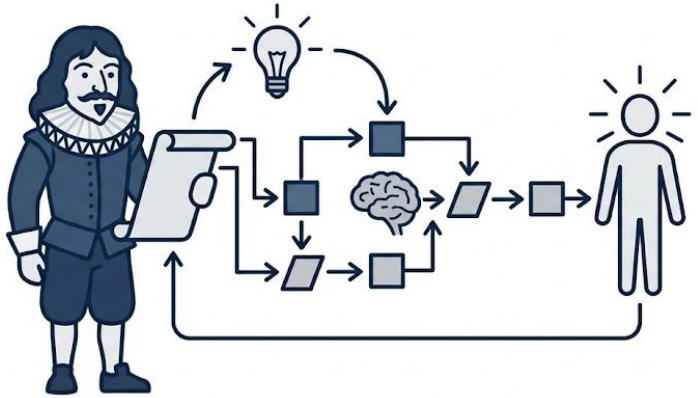
Modelo de Pensamiento Racional

Permite construir un modelo integral de pensamiento: desde la percepción inicial hasta la comprensión del mundo y la realización de predicciones.

De Pensar a Actuar

Solo pensar no basta para generar comportamientos inteligentes; se necesita actuar. El dilema tradicional de *"¿Pienso, luego existo?"* ya no es suficiente.

Pensando Racionalmente



© gonzalezhalexis@gmail.com ©

Inteligencia Artificial

Agentes y Racionalidad

Un agente opera autónomamente, percibe el ambiente, se adapta y cumple objetivos.

Agente Racional: Busca alcanzar el mejor escenario posible o el mejor escenario esperado ante incertidumbre.

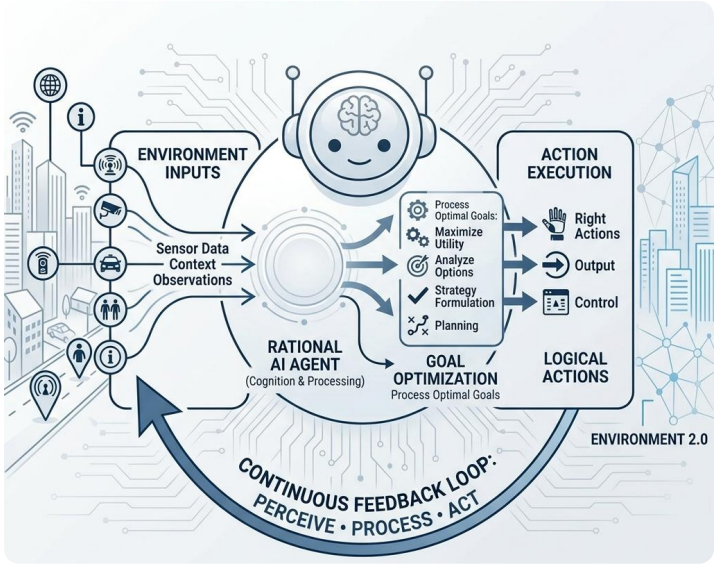
El Modelo Estándar

La IA se ha enfocado en diseñar agentes que hacen "lo correcto", definiendo esto según los objetivos provistos al agente.

Alcance Multidisciplinario

Este modelo no solo predomina en Inteligencia Artificial, sino también en teoría del control, estadística y economía.

Agentes Racionales y Entorno



© gonzalezhalexis@gmail.com ©

Inteligencia Artificial

Máquinas Beneficiosas

El modelo estándar de IA asume que existe un objetivo específico bien definido. Sin embargo, en la vida real resulta sumamente complejo especificar dicho objetivo de manera completa y correcta.

Caso de Estudio: Auto Autónomo

Un vehículo autónomo debe ponderar seguridad, velocidad y confort en entornos dinámicos e impredecibles, ilustrando la dificultad de programar objetivos perfectos.

Entorno de Conducción Autónoma



© gonzalezhalexis@gmail.com ©

Inteligencia Artificial

Alineación de Valores

Lograr un acuerdo entre nuestras preferencias y los objetivos de la máquina es el problema central de alineación. Los objetivos asignados deben alinearse con los humanos.

Laboratorio vs. Vida Real

En el laboratorio basta con resetear el sistema. En el mundo real esto es imposible, por lo que el modelo estándar no es apropiado y requiere una nueva formulación.

Máquinas Cautelosas y Beneficiosas

Al reconocer que no conoce el objetivo completo, la máquina tiene incentivos para actuar con cautela, pedir permiso, aprender de la observación y ceder al control humano.

Control Humano y Alineación



© gonzalezhalexis@gmail.com ©

Inteligencia Artificial

Informática e IA

La Informática provee la infraestructura computacional sobre la cual la IA simula capacidad de razonamiento y aprendizaje.

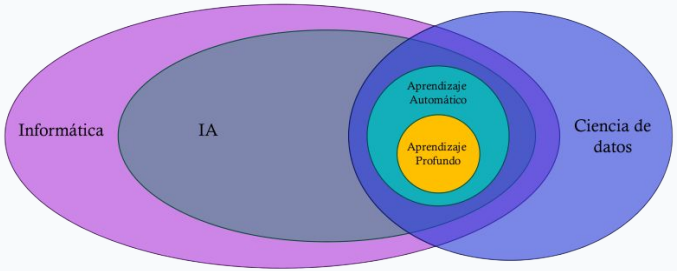
ML y Aprendizaje Profundo

El Aprendizaje Automático identifica patrones en datos, mientras que el Aprendizaje Profundo emplea redes neuronales avanzadas.

Ciencia de Datos

Se interseca con la IA y el ML para extraer conocimiento y valor estadístico a partir de grandes volúmenes de datos.

Campos Conectados e Intersección



© gonzalezhalexis@gmail.com ©

Beneficios de la IA

Superación de Límites

Si la civilización es producto de la inteligencia humana, las máquinas inteligentes pueden elevar ese techo cognitivo.

Automatización de Tareas

Los robots y la IA pueden liberarnos de tareas rutinarias y nimias, permitiendo enfocarnos en actividades de valor.

Aceleración Científica

Impulsa el ritmo de los descubrimientos e investigaciones científicas en múltiples disciplinas.

Nuevas Fronteras

Y más... ¿Qué otros beneficios e impactos futuros se les ocurren?

Aplicaciones en la Vida Real



© gonzalezhalexis@gmail.com ©

Riesgos de la IA

Armas Autónomas
Desarrollo e implementación de armas letales autónomas.

Vigilancia Masiva
Sistemas de vigilancia y persuasión social (estilo 1984).

Sesgos y Discriminación
Toma de decisiones sesgadas por algoritmos opacos.

Impacto en Empleos
Desplazamiento de puestos de trabajo y automatización.

Aplicaciones Críticas
Riesgos en infraestructura, salud y seguridad pública.

Ciberseguridad
Ataques automatizados y nuevas amenazas digitales.

Casos Reales y Evidencias



© gonzalezhalexis@gmail.com ©



02

Main Content

© gonzalezhalexis@gmail.com ©

Historia de la IA

El principio de la inteligencia artificial (1943-1956)

1943 — Primer Trabajo de IA

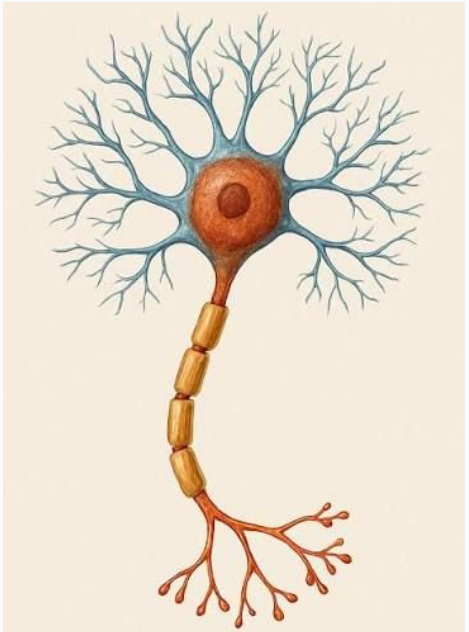
Primer trabajo reconocido de IA desarrollado por Warren McCulloch y Walter Pitts.

Modelo de Neurona Artificial

Cada neurona se prende o apaga al responder a la estimulación de varias neuronas vecinas.

1949 — Donald Hebb & Aprendizaje

Creación de la regla para modificar la intensidad de conexión entre neuronas (aprendizaje), vigente hasta hoy.



© gonzalezhalexis@gmail.com ©

Historia de la IA

El principio de la inteligencia artificial (1943-1956)

1947 — Primeras Lecciones de IA

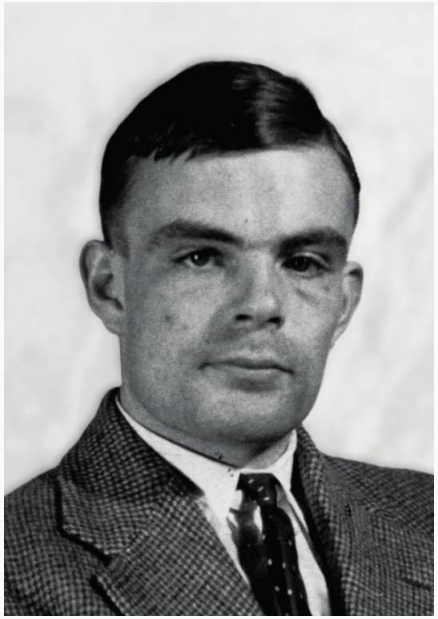
Alan Turing dio sus primeras lecciones en IA en 1947.

Conceptos Clave e Innovaciones

Introdujo el test de Turing, aprendizaje automático, algoritmos genéticos y aprendizaje por refuerzo.

Visión del Aprendizaje Automático

Sugirió que sería más fácil crear IA desarrollando algoritmos de aprendizaje y enseñando a la máquina en lugar de programarla a mano.



© gonzalezhalexis@gmail.com ©

Historia de la IA

Entusiasmo inicial, grandes expectativas (1952-1969)

Demostrando la Capacidad de la IA

Frente al escepticismo que afirmaba "una máquina nunca podrá hacer X", los investigadores demostraron lo contrario en juegos, rompecabezas, matemáticas y pruebas de IQ.

1956 — Software Autónomo (Damas)

Arthur Samuel desarrolló un programa para jugar a las damas usando aprendizaje por refuerzo. El software aprendió y mejoró su juego por sí solo.



© gonzalezhalexis@gmail.com ©

Historia de la IA

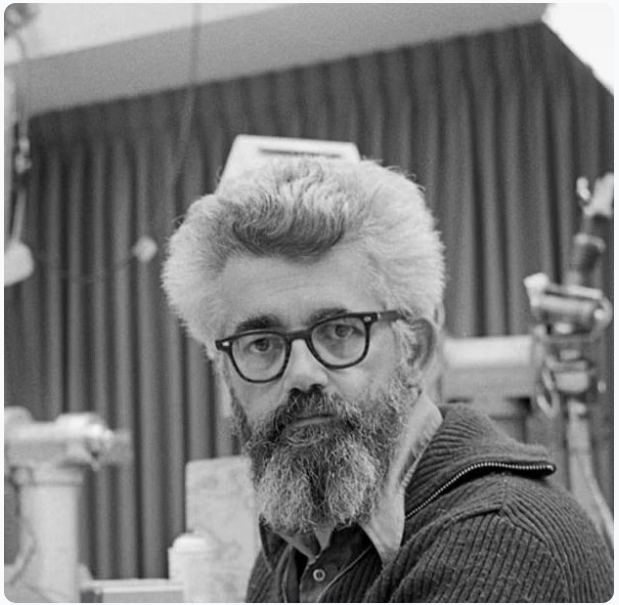
Entusiasmo inicial, grandes expectativas (1952-1969)

1958 — Lisp (John McCarthy)

Creado por John McCarthy, se convirtió en el lenguaje de programación estándar para la investigación en Inteligencia Artificial durante más de 30 años.

Advice Taker

Programa hipotético diseñado para encarnar el conocimiento general del mundo y derivar planes de acción. Diseñado para incorporar nuevos axiomas sin ser reprogramado.



© gonzalezhalexis@gmail.com ©

Historia de la IA

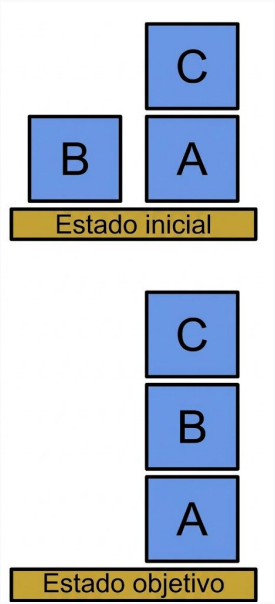
Entusiasmo inicial, grandes expectativas (1952-1969)

Micro Mundos (1963–1969)

Marvin Lee Minsky en el MIT supervisó a estudiantes que desarrollaron software en dominios limitados, llamados micro mundos.

Mundo de Bloques

El objetivo es construir pilas verticales moviendo un bloque a la vez (a la mesa o sobre otro). Los bloques cubiertos no se pueden mover y algunos no permiten apilar encima.



© gonzalezhalexis@gmail.com ©

Historia de la IA

Primer invierno (1966-1973)

Sobre-expectativas de Herbert Simon (1957)

Predijo que en 10 años la IA lograría batir al campeón mundial de ajedrez y resolver teoremas complejos... cosas que demoraron 40 años.

Introspección humana

Los algoritmos de ese momento se basaban en introspección informada en cómo los humanos realizan una tarea.

Complejidad algorítmica

No se habían desarrollado las teorías computacionales de complejidad algorítmica, por lo que no se sabía cómo escalaban los algoritmos.



© gonzalezhalexis@gmail.com ©

Historia de la IA

Primer invierno (1966-1973)

El libro "Perceptrones" (1969)

Minsky y Papert probaron que los perceptrones (modelo de neurona) solo podían representar funciones simples, fallando en la función lógica XOR.

Impacto e invierno de la IA

Esta limitación percibida congeló casi por completo el desarrollo del Deep Learning y la neurociencia computacional durante 15 años.

Aclaración posterior (Spoiler)

Más adelante se demostró que la crítica aplicaba solo a perceptrones simples de una capa, pero no era válida para redes multicapa.



© gonzalezhalexis@gmail.com ©

Historia de la IA

Sistemas expertos (1969-1986)

Primera aplicación exitosa

Ante la imposibilidad de escalar de los algoritmos previos, surgieron los sistemas expertos como el primer gran éxito de la IA.

El programa DENDRAL (1969)

Infería estructuras moleculares introduciendo reglas de expertos químicos para evitar búsquedas innecesarias.

Invierno comercial de la IA

Tras la adopción inicial de los sistemas expertos, el campo enfrentó un nuevo periodo de estancamiento comercial.



© gonzalezhalexis@gmail.com ©

Historia de la IA

El retorno de las redes neuronales (1986-presente)

Algoritmo Back-propagation

En los 80, se re-descubrió el algoritmo de aprendizaje back-propagation, tarea fundamental para que las redes se entrenaran.

Geoff Hinton y el «éter de la IA»

Geoff Hinton, uno de los principales impulsores de la vuelta de redes neuronales, describió a los símbolos como el “éter de la IA”.

Revisión de la IA simbólica

Fue la primera vez que se puso en discusión que la inteligencia se trataba únicamente de símbolos representativos.



© gonzalezhalexis@gmail.com ©

Historia de la IA

Aprendizaje Automático (1987-presente)

Enfoque más científico

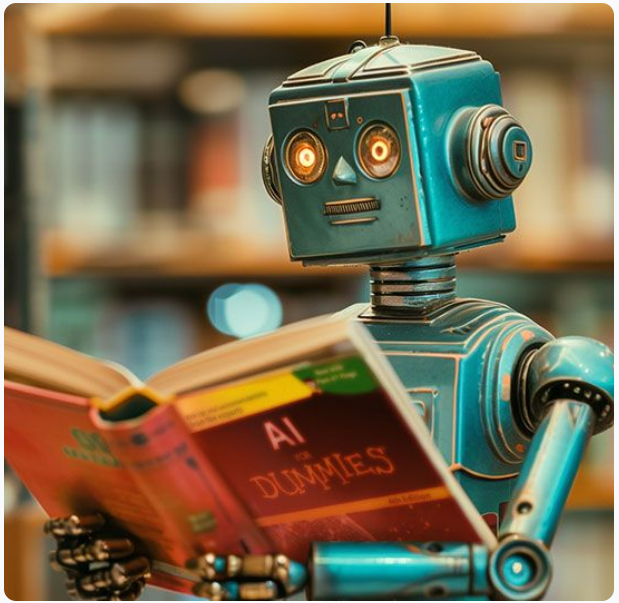
A finales de los '80, la IA se alejó de la lógica booleana para adoptar modelos probabilísticos y experimentos validables.

Conexión multidisciplinaria

Reincorporó disciplinas clave como estadística, teoría de control, teoría de la información y optimizaciones.

Prevalencia del Aprendizaje Automático

Modelos como cadenas de Markov, redes bayesianas y SVM superaron a las redes neuronales con menor cómputo.



© gonzalezhalexis@gmail.com ©

Historia de la IA

Big data (2001-presente)

Explosión de datos

Con la llegada de la World Wide Web y la Ley de Moore, nacieron datasets enormes: el fenómeno Big Data.

Nuevos algoritmos

Surgió la necesidad de desarrollar nuevos algoritmos diseñados para tomar ventaja de este gran volumen de datos.

IBM Watson y Jeopardy! (2011)

La IA recuperó atractivo comercial. En 2011, IBM Watson alcanzó el nivel de campeón humano en el programa Jeopardy!



© gonzalezhalexis@gmail.com ©

Historia de la IA

Deep Learning (2001-presente)

Redes Neuronales Profundas

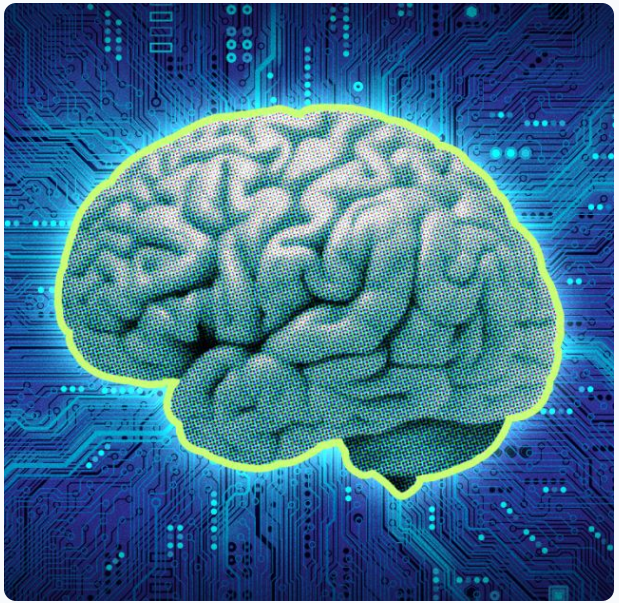
Con la llegada de Big Data y suficiente capacidad de procesamiento, las redes neuronales lograron explotar su máximo potencial imitando estructuras nerviosas.

Potencia de Cómputo y Hardware

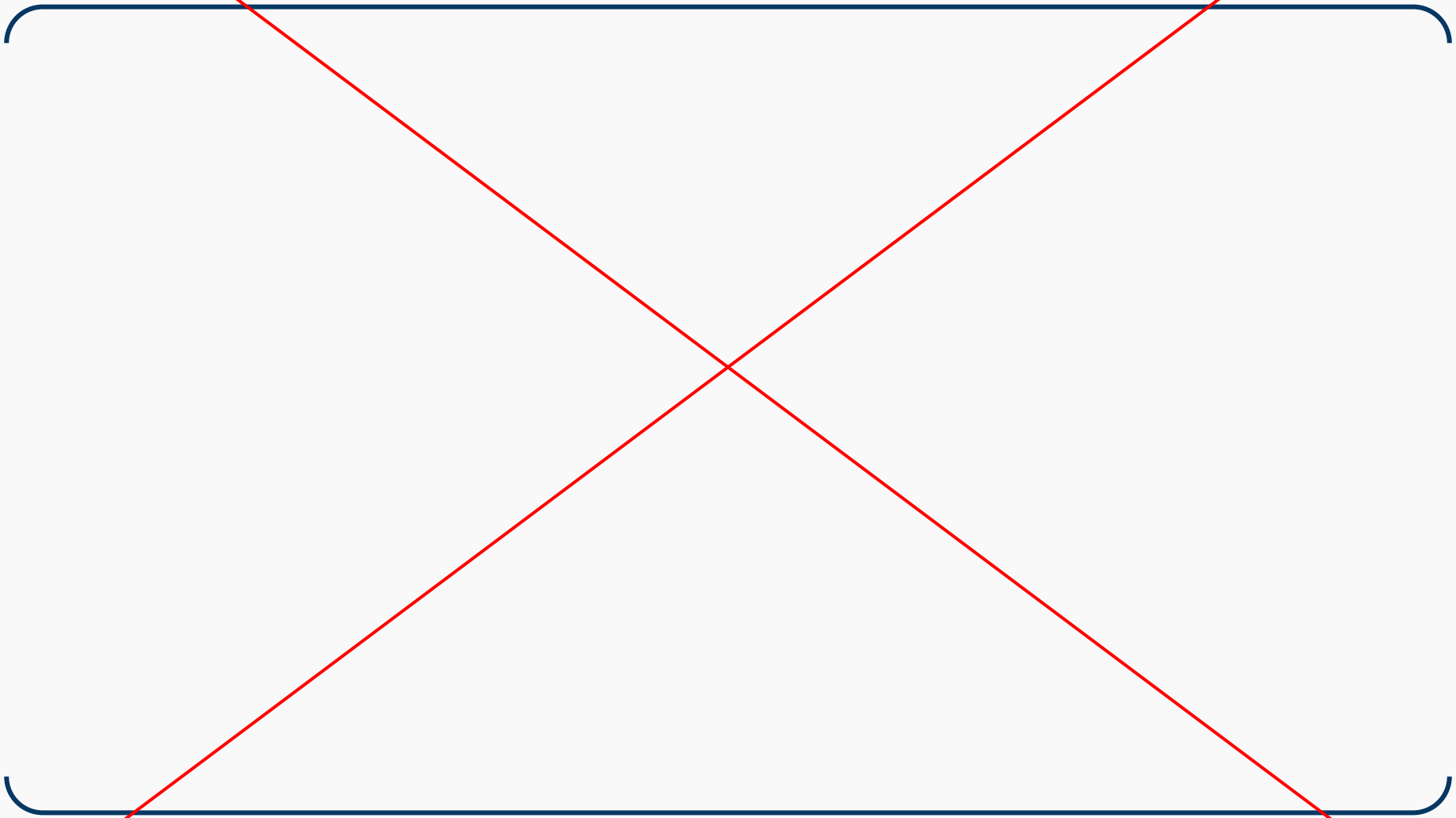
Surgimiento de CPUs de alta potencia (10^{10} ops/s) y hardware dedicado (GPU, TPU, FPGA) para procesamiento paralelo de tensores (10^{17} ops/s).

Entrenamiento con Big Data

Disponibilidad de petabytes de datos para el entrenamiento de modelos, impulsando avances disruptivos en múltiples campos de aplicación.

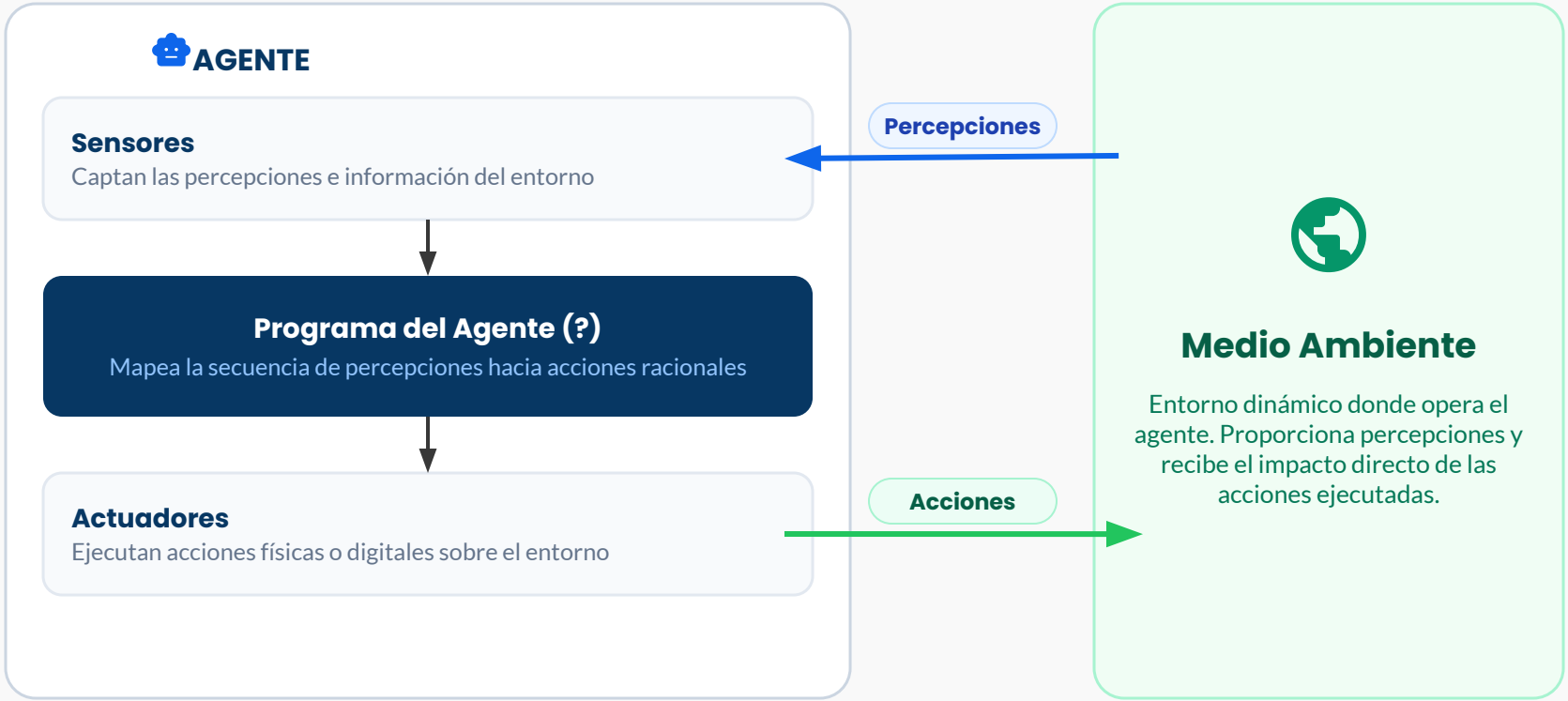


© gonzalezhalexis@gmail.com ©


© gonzalezhalexis@gmail.com ©

Agentes Racionales

Arquitectura e interacción entre el Agente y su Medio Ambiente



© gonzalezhalexis@gmail.com ©

Agentes Racionales

Fundamentos de la Toma de Decisiones en Agentes Inteligentes

Percepción (Percept)

Entrada sensorial puntual que el agente recibe del entorno en un instante específico del tiempo.

EJEMPLO EN EL ENTORNO

Captura individual: Identificación de un objeto o flor en instante t.

Secuencia de Percepciones

Historial completo y cronológico de todo lo que el agente ha percibido a lo largo de su operación.

SECUENCIA HISTÓRICA ACUMULADA

[Percepción 1] → [Percepción 2] → ... → [Percepción N]

Mapeo Racional

La decisión o acción seleccionada por el agente depende explícitamente de la **secuencia completa de percepciones** acumulada hasta el momento, no solo del estado actual.

Una elección de acción de un agente en un momento dado puede depender en su conocimiento incorporado y en la secuencia completa de percepciones hasta ese instante, pero no en cualquier cosa que no haya percibido.

© gonzalezhalexis@gmail.com ©

Agentes Racionales

Distinción entre la Caracterización Matemática y la Implementación Física

El Comportamiento del Agente

En términos matemáticos, el comportamiento viene dado por la **función del agente** que mapea una percepción dada en una acción. Con tiempo infinito, se podría construir una tabla que tabule cada acción ante cualquier secuencia de percepción.

Función del Agente

DESCRIPCIÓN MATEMÁTICA ABSTRACTA

- Caracterización **externa** del comportamiento del agente.
- Mapea cualquier secuencia de percepciones hacia una acción concreta.
- Concepto abstracto formal independiente de la arquitectura o hardware.

Programa del Agente

IMPLEMENTACIÓN CONCRETA

- Implementación **interna** de la función del agente.
- Código o algoritmo real que se ejecuta en el agente artificial.
- Opera dentro de una arquitectura física o sistema de computación.

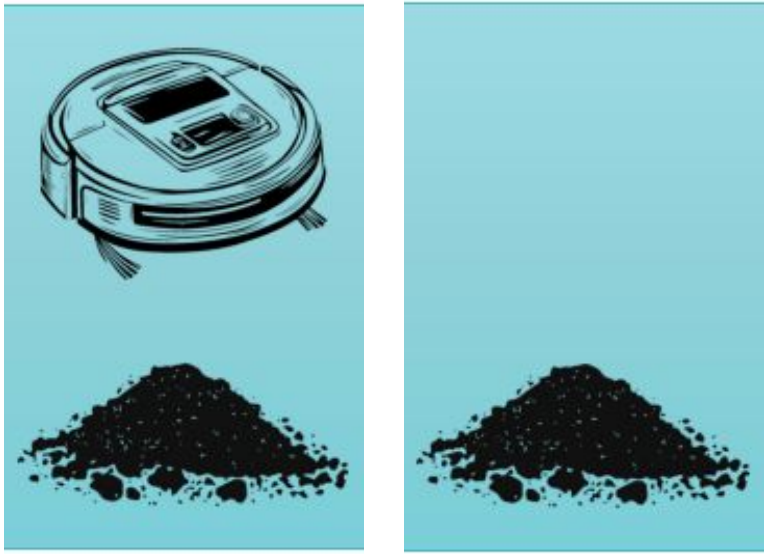
© gonzalezhalexis@gmail.com ©

Agentes Racionales

Modelo del Agente Aspiradora: Estado del Entorno, Percepciones y Acciones

Entorno de Trabajo

Mundo de dos ubicaciones (A y B)



Percepción (Percepts)

Estado de la casilla actual percibido por el agente

¿Donde estoy? ¿Cómo está el estado de la casilla actual?

✨ Espacio limpio 💩 Espacio sucio

Acciones (Actions)

Conjunto de operaciones ejecutables en el entorno

⬅️ Mover a la izquierda

➡️ Mover a la derecha

🧹 Limpiar el espacio

⏸️ No hacer nada

© gonzalezhalexis@gmail.com ©

Agentes Racionales

Modelo del Agente Aspiradora: Tabla parcial de secuencia de percepción y acción

Secuencia de Percepción (Percept Sequence)	Acción (Action)
`[A, Limpio]`	➡️ **Derecha**
`[A, Sucio]`	🧹 **Limpiar**
`[B, Limpio]`	⬅️ **Izquierda**
`[B, Sucio]`	🧹 **Limpiar**
`[A, Limpio], [A, Limpio]`	➡️ **Derecha**
`[A, Limpio], [A, Sucio]`	🧹 **Limpiar**
…	…
`[A, Limpio], [A, Limpio], [A, Limpio]`	➡️ **Derecha**
`[A, Limpio], [A, Limpio], [A, Sucio]`	🧹 **Limpiar**

© gonzalezhalexis@gmail.com ©

Agentes Racionales

Modelo del Agente Aspiradora: Programa de agente reflexivo en ambiente de dos casillas

Código del Agente

Función REFLEX_VACUUM_AGENT en Python

```
def REFLEX_VACUUM_AGENT(location,
status) -> Action:
  if status == "Dirty":
    return Suck
  elif location == A:
    return Right
  else: # location == B
    return Left
```

Reglas de Decisión

Flujo lógico de acciones según la percepción

1. ¿Estado es "Sucio"?

🧹 Acción: Suck (Limpiar el espacio)

2. ¿Ubicación actual es A? (Limpia)

➡️ Acción: Right (Mover a casilla B)

3. ¿Ubicación actual es B? (Limpia)

⬅️ Acción: Left (Mover a casilla A)

© gonzalezhalexis@gmail.com ©

Agentes Racionales

Concepto clave y paradoja en el comportamiento del agente

🎯 Definición de Medida de Rendimiento

Incluye los criterios objetivos que determinan el éxito en el comportamiento del agente. En general, son establecidos directamente por el diseñador.

⚠️ El Riesgo de Métricas Mal Diseñadas

Ejemplo de la Aspiradora: Si definimos la medida como *"cantidad de suciedad limpiada en 8 horas"*...

Un agente racional buscará maximizar la métrica tirando la basura al suelo para volver a limpiarla repetidamente en bucle.

💡 Conclusión Clave

La selección de la medida de rendimiento adecuada no es siempre fácil. Debe evaluar el estado real del entorno, no solo las acciones.



© gonzalezhalexis@gmail.com ©

Agentes Racionales

Medida de rendimiento y dilemas de evaluación

💡 Regla General de Diseño

Es mejor diseñar medidas de utilidad de acuerdo con lo que se quiere para el entorno, más que de acuerdo con cómo se cree que el agente debe comportarse.

📌 **Ejemplo:** Recompensar al agente con 1 punto por cada cuadrícula limpia en cada periodo de tiempo.

⚖️ Dimensión Filosófica

Determinar cuál es la forma más adecuada de evaluar la efectividad del agente es, en última instancia, una cuestión filosófica de diseño de IA.

⚠️ El Dilema de la Medida Promedio

La noción de suelo limpio se basa en un nivel de limpieza promedio a lo largo del tiempo. Esto puede alcanzarse mediante dos comportamientos radicalmente distintos:

1. Limpieza Mediocre

Haciendo una limpieza mediocre pero continua.

2. Limpieza Intermitente

Limpiando en profundidad, con largos descansos.



© gonzalezhalexis@gmail.com ©

Agentes Racionales

Racionalidad

⚙️ 4 Factores de la Racionalidad

La racionalidad en un momento dado depende exclusivamente de:

1. Medida de Rendimiento
Criterio objetivo que define el éxito.

2. Conocimiento Previo
Información acumulada sobre el entorno.

3. Acciones Posibles
Acciones que el agente puede ejecutar.

4. Secuencia de Percepciones
Historial de percepciones hasta el momento.

💡 Definición de Racionalidad

En cada posible secuencia de percepciones, un **agente racional** deberá seleccionar aquella acción que supuestamente **maximice su medida de rendimiento**.

Esta decisión se basa en las **evidencias aportadas por la secuencia de percepciones** y en el **conocimiento que el agente mantiene almacenado**.

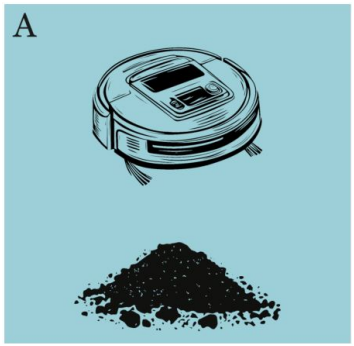


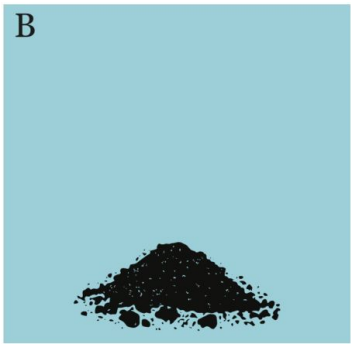
© gonzalezhalexis@gmail.com ©

Agentes Racionales

Racionalidad

🤖 Entorno y Código del Agente






```python
def REFLEX_VACUUM_AGENT(location, status) -> Action:
    if status == "Dirty":
        return Suck
    elif location == A:
        return Right
    else: # location == B
        return Left
```

⚖️ ¿Es Racional? Depende

Asumamos las siguientes condiciones:

- **Rendimiento:** 1 punto por espacio limpio en 10.000 pasos de vida.
- **Entorno:** Geografía conocida *a priori*, suciedad desconocida. Lo limpio no se vuelve a ensuciar y el movimiento es 100% preciso.
- **Acciones:** Solo 3 disponibles (*Derecha*, *Izquierda*, *Limpiar*).
- **Percepción:** Percibe siempre bien su ubicación y estado del piso.

✅ Conclusión:

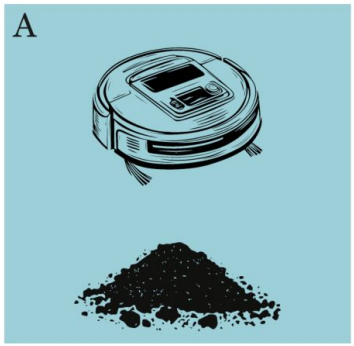
Bajo estas circunstancias específicas, la función del agente **es racional**.

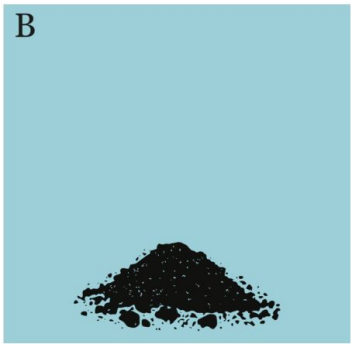
© gonzalezhalexis@gmail.com ©

Agentes Racionales

Racionalidad

🤖 Entorno y Código del Agente






```python
def REFLEX_VACUUM_AGENT(location, status) -> Action:
    if status == "Dirty":
        return Suck
    elif location == A:
        return Right
    else: # location == B
        return Left
```

⚖️ ¿Es Racional? Depende

⚠️ Condición de Irracionalidad:

Al agregar una penalización de 1 punto por movimiento, el agente entra en un bucle y queda oscilando.

💡 Soluciones y Consideraciones:

1. Incorporar Memoria

Detener la aspiradora al finalizar la limpieza completa.

2. Rutinas de Chequeo

Si el piso se puede volver a ensuciar con el tiempo.

3. Algoritmo de Búsqueda

Necesario para explorar cuando la geografía es desconocida.

© gonzalezhalexis@gmail.com ©

Agentes Racionales

Especificación del Entorno de Trabajo (PEAS)

Para definir un agente racional se deben especificar las medidas de rendimiento, entorno, actuadores y sensores (PEAS):

P Performance

Medida de desempeño

Ejemplo: Aterrizar < 10cm de error

E Environment

Entorno

Ejemplo: Cubierta en movimiento, viento lateral, etc.

A Actuators

Actuadores

Ejemplo: Motores brushless, servo de cámara, etc.

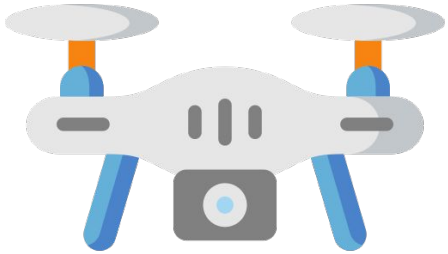
S Sensors

Sensores

Ejemplo: GPS, giroscopio, acelerómetro, etc.

Ejemplo: Dron VTOL

"Vertical Take-Off and Landing"



💡 Pensar un sistema con **PEAS** obliga a aclarar **qué es el éxito** antes de ponerse a programar.

© gonzalezhalexis@gmail.com ©

Agentes Racionales

Especificación del Entorno de Trabajo (PEAS)

Agente	Performance (P)	Environment (E)	Actuators (A)	Sensors (S)
Sistema de diagnóstico médico	Paciente sano, reducir costos	Paciente, personal del hospital	Display de preguntas, test, diagnosis y tratamientos	Pantalla táctil / entrada por voz
Sistema de análisis de imágenes de satélite	Categorización correcta de objetos, terreno	Satélite en órbita, enlace, clima	Visualización de categorización de escenas	Cámara digital de alta resolución
Robot levanta piezas	Porcentaje de piezas en contenedores correctos	Cinta transportadora con piezas, contenedores	Brazo y mano articulados	Sensores de cámara, táctiles y de ángulo articular

© gonzalezhalexis@gmail.com ©

Agentes Racionales

Especificación del Entorno de Trabajo | Dimensiones de Categorización

Veamos algunas dimensiones con las que podemos categorizar los entornos de trabajo:

1 Observabilidad

Totalmente observable vs. Parcialmente observable

Acceso a sensores del estado completo.

2 Determinismo

Deterministas vs. Estocástico

Certeza o aleatoriedad en los cambios.

3 Secuencialidad

Episódico vs. Secuencial

Independencia entre decisiones tomadas.

4 Dinamismo

Estático vs. Dinámico

Cambio continuo del entorno al deliberar.

5 Continuidad

Discreto vs. Continuo

Naturaleza finita o continua de estados.

6 Agentes

Agente individual vs. Multiagente

Competitivo o cooperativo.

💡 La complejidad en la construcción de un agente inteligente depende directamente de esta combinación de **propiedades del entorno**.

© gonzalezhalexis@gmail.com ©

Agentes Racionales

Propiedades del Entorno de Trabajo | Observabilidad

Totalmente Observable

Los sensores del agente proporcionan acceso al **estado completo** del entorno en cada punto del tiempo.

Ejemplo

Resolver Sudoku con visión

El agente percibe la totalidad del tablero y todas las celdas de forma clara (estado completo).

Parcialmente Observable

El agente tiene **información limitada o incompleta** del entorno debido a ruido, limitaciones o datos faltantes.

Ejemplo

Chatbot sin memoria de conversación

Solo ve el mensaje actual recibido, sin acceso al historial previo del contexto.

© gonzalezhalexis@gmail.com ©

Agentes Racionales

Propiedades del Entorno de Trabajo | Determinismo

Determinista

Misma entrada → **siempre la misma salida** (no hay azar).

Ejemplo

Scripts fijos en videojuegos

Agente que ejecuta secuencias de instrucciones predeterminadas e invariables.

Estocástico

Misma entrada → la salida puede **variar según una probabilidad** (hay azar/ruido).

Ejemplo

Trading bot financiero

Bot con precios de mercado modelados mediante procesos aleatorios y probabilísticos.

© gonzalezhalexis@gmail.com ©

Agentes Racionales

Propiedades del Entorno de Trabajo | Episodicidad

Episódico

La experiencia se divide en **episodios independientes**; la acción presente no afecta las decisiones futuras.

Ejemplo

Clasificador de imágenes médico

Cada estudio de un paciente se procesa de forma totalmente independiente del anterior.

Secuencial

Cada decisión afecta al estado actual y **condiciona las acciones futuras** de forma continua.

Ejemplo

Agente de RL que navega un laberinto

Cada movimiento altera la posición del agente y determina sus opciones e interacciones posteriores.

© gonzalezhalexis@gmail.com ©

Agentes Racionales

Propiedades del Entorno de Trabajo | Dinamismo

Estático

El entorno permanece **sin cambios** mientras el agente está deliberando o respondiendo.

Ejemplo

LLM respondiendo una consulta

Responde a una pregunta fija sin que existan modificaciones en el entorno durante el proceso.

Dinámico

El entorno **puede cambiar continuamente** mientras el agente se encuentra deliberando.

Ejemplo

Asistente de agendamiento

Agenda reuniones de forma interactiva mientras otros usuarios editan el calendario en paralelo.

© gonzalezhalexis@gmail.com ©

Agentes Racionales

Propiedades del Entorno de Trabajo | Continuidad y Discreción

Discreto

El estado y las acciones del entorno se dividen en **pasos y valores claramente delimitados**.

Ejemplo

Agente RL en tablero (ajedrez)

Interacción estructurada con acciones por turnos en un conjunto finito de estados.

Continuo

Las magnitudes físicas y decisiones se evalúan en un **rango continuo e ininterrumpido**.

Ejemplo

Control de dron

Ajustes constantes en tiempo real de acciones continuas de throttle y ángulos.

© gonzalezhalexis@gmail.com ©

Agentes Racionales

Propiedades del Entorno de Trabajo | Número de Agentes

Agente individual

Un solo agente opera de forma autónoma en el entorno.

Ejemplo

Recomendador de productos

Decide qué ítem mostrar al usuario de forma independiente.

Multiagente competitivo

Múltiples agentes compiten entre sí optimizando metas opuestas.

Ejemplo

Subastas y StarCraft

Bots de negociación en subastas y equipos con oponentes IA.

Multiagente cooperativo

Múltiples agentes colaboran para alcanzar un objetivo compartido.

Ejemplo

Flota de robots

Robots de almacén coordinados repartiendo tareas.

© gonzalezhalexis@gmail.com ©

Programa de los Agentes

Tipos de Programas de Agentes

01

Reactivos Simples

Actúan solo con base en la percepción actual.

02

Basados en Modelos

Mantienen un estado interno del mundo.

03

Basados en Objetivos

Actúan para alcanzar metas deseables.

04

Basados en Utilidad

Maximizan una función de preferencia.

05

Que Aprenden

Mejoran con la experiencia de forma autónoma.

© gonzalezhalexis@gmail.com ©

Programa de los Agentes

Agentes reactivos simples • Arquitectura e Implementación

simple_reflex_agent.py

```
def SimpleReflexAgentProgram(rules, interpret_input):

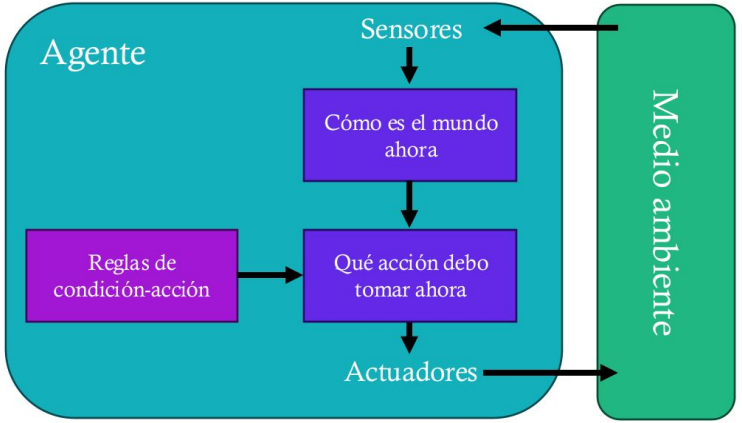
    def program(percept):
        state = interpret_input(percept)
        rule = rule_match(state, rules)
        action = rule.action
        return action

    return program
```

Lógica de Funcionamiento

- Toma decisiones basadas **únicamente en la percepción actual**.
- Utiliza un conjunto de reglas **condición-acción** (if-then).
- Ignora el historial completo de percepciones previas.

Esquema del Entorno y Agente



Características del Modelo

- Conexión directa entre **Sensores** y **Actuadores**.
- Eficaz únicamente en entornos **completamente observables**.

© gonzalezhalexis@gmail.com ©

Programa de los Agentes

Agentes reactivos basados en modelos • Arquitectura e Implementación

model_based_reflex_agent.py

```
def ModelBasedReflexAgentProgram(rules, update_state, model):

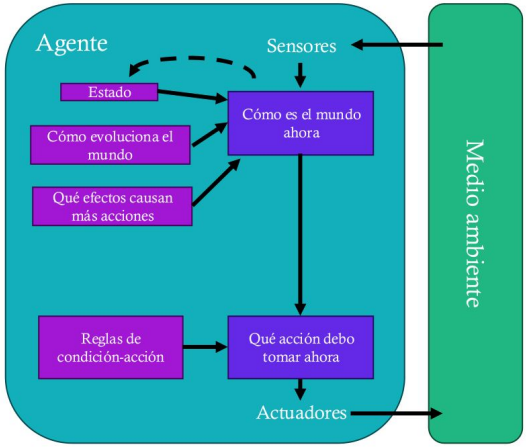
    def program(percept):
        program.state = update_state(program.state,
                                     program.action,
                                     percept, model)
        rule = rule_match(program.state, rules)
        action = rule.action
        return action

    program.state = program.action = None
    return program
```

Lógica de Funcionamiento

- Mantiene un **estado interno** para rastrear el mundo.
- Combina el estado actual con un **modelo del entorno**.
- Evalúa reglas condición-acción con memoria de percepciones.

Esquema del Entorno y Agente



Características del Modelo

- Rastrea aspectos del entorno no percibidos directamente.
- Eficaz en entornos **parcialmente observables**.

© gonzalezhalexis@gmail.com ©

Programa de los Agentes

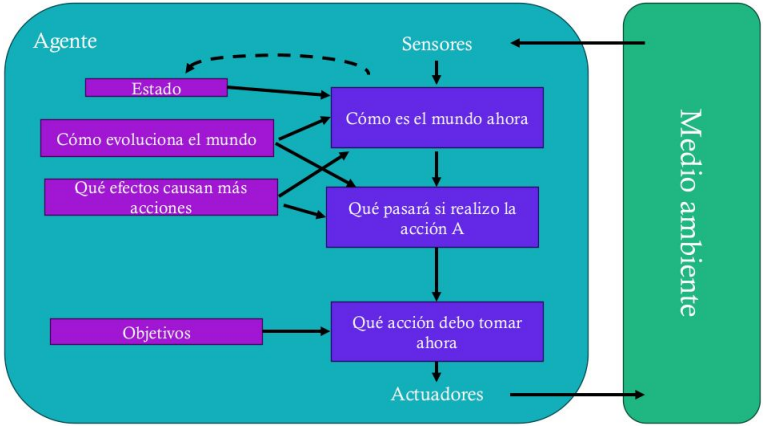
Agentes basados en objetivos • Arquitectura e Implementación

Lógica de Funcionamiento

Proceso de Decisión

- Mantiene un **estado interno** que combina la percepción actual y la evolución del mundo.
- Evalúa las consecuencias: **"¿Qué pasará si realizo la acción A?"**
- Combina la previsión con **Objetivos explícitos** para elegir la mejor opción.
- Soporta búsqueda y planificación para alcanzar metas complejas.

Esquema del Agente Basado en Objetivos



Características del Modelo

- Mayor **flexibilidad**: el comportamiento cambia fácilmente adaptando las metas.
- Combina el estado del entorno con la deseabilidad de los estados futuros.

© gonzalezhalexis@gmail.com ©

Programa de los Agentes

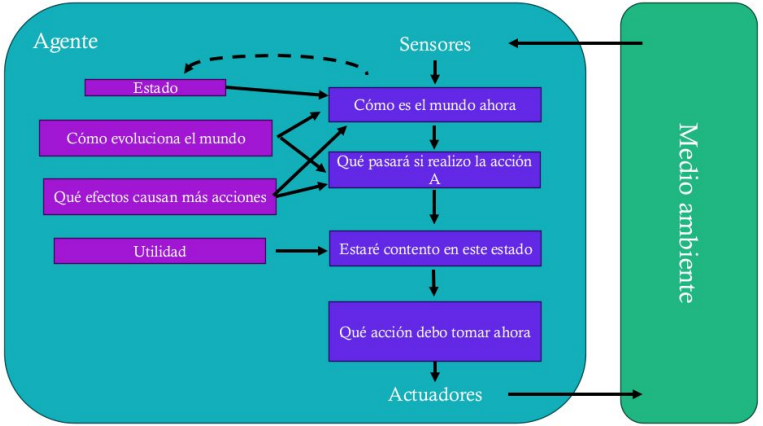
Agentes basados en utilidad • Arquitectura e Implementación

Lógica de Funcionamiento

Función de Utilidad y Decisión

- Mantiene un **estado interno** y prevé la evolución del entorno.
- Evalúa múltiples alternativas: **"¿Estaré contento en este estado?"**
- Aplica una **métrica cuantitativa** (utilidad) para medir el grado de éxito o satisfacción.
- Permite tomar decisiones óptimas ante objetivos contrapuestos o incertidumbre.

Esquema del Agente Basado en Utilidad



Características del Modelo

- Proporciona un trade-off continuo entre metas en vez de un resultado binario.
- Permite la toma de decisiones racional bajo **incertidumbre**.

© gonzalezhalexis@gmail.com ©

Programa de los Agentes

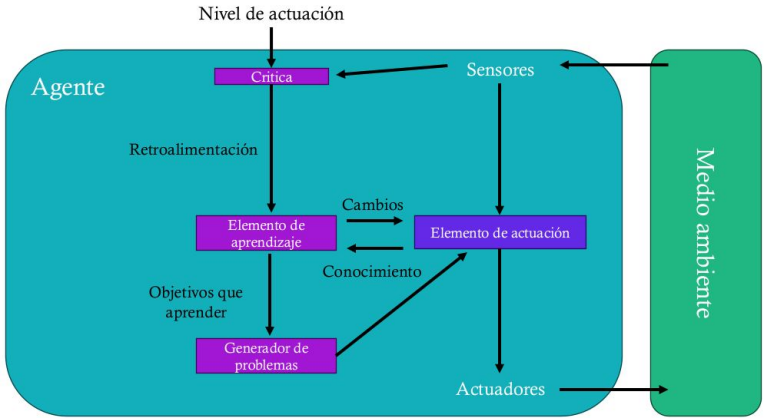
Agentes que aprenden • Arquitectura e Implementación

Lógica de Funcionamiento

Componentes Clave

- **Crítica:** Evalúa el desempeño según un criterio fijo externo.
- **Elemento de aprendizaje:** Modifica el conocimiento para mejorar en el futuro.
- **Generador de problemas:** Sugiere acciones para obtener nuevas experiencias.
- **Elemento de actuación:** Selecciona las acciones a tomar en el entorno.

Esquema del Agente que Aprende



Características del Modelo

- Permite operar en **entornos desconocidos** e ir mejorando progresivamente.
- Separa el juicio de calidad (crítica) del proceso de elección de acciones.

© gonzalezhalexis@gmail.com ©

Programa de los Agentes

Ejemplo 1: Termostato • Agente Basado en Reglas

Diseñaremos un termostato que mantiene una habitación a temperatura confortable.

Especificación PEAS



P: Performance
Mantener temp. 20-22°C, minimizar cambios

E: Environment
Habitación con temperatura variable

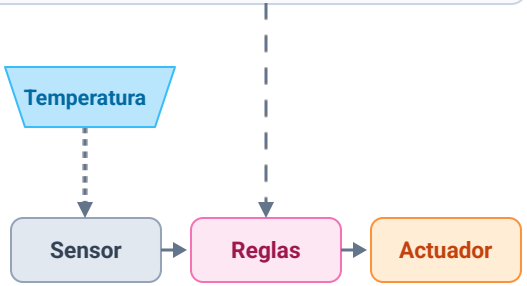
A: Actuators
Calefacción (ON/OFF), Aire acondicionado (ON/OFF)

S: Sensors
Sensor de temperatura

Reglas y Arquitectura

Reglas del agente:

```
SI temp < 20°C
ENTONCES encender_calefaccion()
SI temp > 22°C
ENTONCES encender_aire_acondicionado()
SI 20°C <= temp <= 22°C
ENTONCES apagar_todo()
```



Características del Entorno

Observable
El agente ve la temperatura actual

Determinista
Acciones con efectos predecibles

Episódico
Cada decisión es independiente

Estático
Cambia solo por acciones del agente

Discreto
Lectura en valores discretos

Agente único
Opera solo un termostato

© gonzalezhalexis@gmail.com ©

Programa de los Agentes

Ejemplo 1: Termostato • Preguntas para Reflexionar

1. Temperatura Muy Baja

¿Qué pasa si la temperatura ambiente es muy baja?

El agente trabajará constantemente para intentar compensar la pérdida de calor.



2. Evaluación de Racionalidad

¿Es este agente racional?

Sí. Maximiza la medida de desempeño esperada según sus reglas y la información sensorial disponible en su entorno.

3. Limitaciones del Agente

¿Qué limitaciones tiene?

- **No puede anticipar** cambios climáticos.
- **No recuerda el pasado** (agente reactivo).
- **Puede oscilar** continuamente en el límite del umbral.

4. Propuestas de Mejora

¿Cómo mejoraría su eficiencia?

- **Agregar histéresis:** definir una zona muerta de tolerancia.
- **Predecir tendencias:** evaluar la velocidad de cambio de temp.
- **Aprendizaje de patrones:** incorporar memoria de hábitos/clima.

© gonzalezhalexis@gmail.com ©

Programa de los Agentes

Ejemplo 2: Agente Aspiradora • Múltiples Percepciones

Extiende el agente reactivo simple para manejar múltiples percepciones y acciones.

Especificación PEAS



P: Performance

Maximizar limpieza, minimizar movimientos

E: Environment

Dos habitaciones (A, B) con suciedad

A: Actuators

Mover_izquierda, Mover_derecha, Aspirar, No_hacer_nada

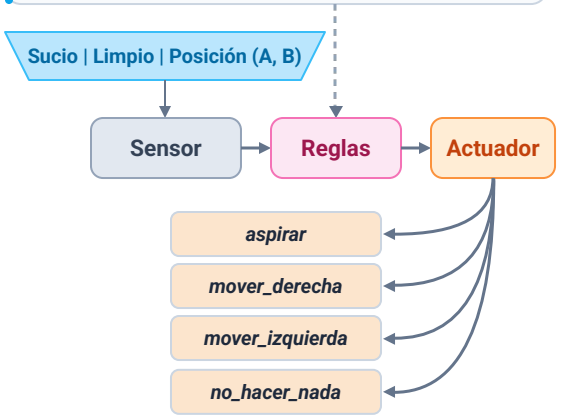
S: Sensors

Posición actual, ¿Hay suciedad aquí?

Reglas y Arquitectura

Reglas del agente:

```
SI hay_suciedad_aqui ENTONCES aspirar
SI posición == A Y habitacion_A_limpia
ENTONCES mover_derecha
SI posicion == B Y habitacion_B_limpia
ENTONCES mover_izquierda
SI todo_limpio ENTONCES no_hacer_nada
```



Características del Entorno

Observable
El agente ve su posición y si hay suciedad

Determinista
Las acciones tienen efectos predecibles

Secuencial
Las decisiones previas afectan el futuro

Estático
Cambia solo por acciones del agente

Discreto
Posiciones y estados discretos

Agente único
Una sola aspiradora

© gonzalezhalexis@gmail.com ©

Programa de los Agentes

Ejemplo 2: Agente Aspiradora • Preguntas para Reflexionar

1. Estados Posibles del Mundo

¿Cuántos estados posibles hay en este mundo?

2 habitaciones × 2 estados × 2 posiciones = **8 estados**



2. Evaluación de Racionalidad

¿Es racional este agente?

Parcialmente: limpia todo, pero luego oscila.

3. Propuesta de Mejora

¿Cómo mejoraríamos el agente?

Necesitamos **memoria** (estado interno).

4. Escalamiento (3 Habitaciones)

¿Qué pasa con 3 habitaciones?

- Las reglas simples se vuelven complejas
- Necesitamos arquitecturas más sofisticadas

© gonzalezhalexis@gmail.com ©

Programa de los Agentes

Ejemplo 3: Agente con Modelo del Mundo • Estado Interno

Introduce agentes que mantienen memoria, planifican acciones y evitan comportamientos redundantes.

Especificación PEAS y Capacidades

P: Performance
Maximizar limpieza, minimizar movimientos innecesarios

E: Environment
Dos habitaciones (A, B) con suciedad

A: Actuators
Mover_izq, Mover_der, Aspirar, No_hacer_nada

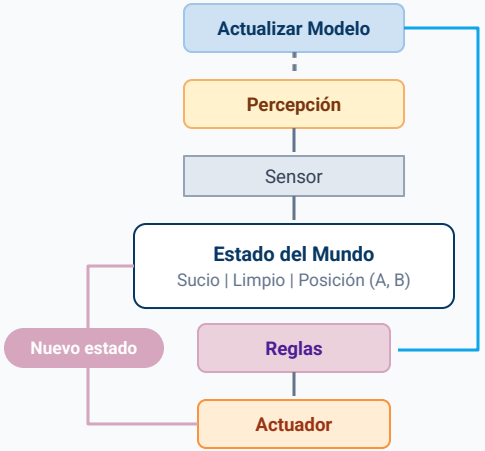
S: Sensors
Posición actual, ¿Hay suciedad aquí?

Ventajas Clave:

- Mantener estado interno (memoria)
- Construir un modelo del mundo
- Planificar acciones futuras
- Evitar oscilaciones redundantes

Arquitectura y Estado Interno

Diferencia Clave
Reactivo Simple: Solo percepción actual
Con Modelo: Percepción + Estado + Modelo del Mundo



Reglas del Agente

```
SI hay_suciedad_aqui
ENTONCES aspirar
Y marcar limpia(pos)
SI todas_habitaciones_limpias
ENTONCES no hacer nada
SI habitacion_no_visitada
ENTONCES ir a(no visitada)
SI pos == A Y hab_B_sucia
ENTONCES mover derecha
SI pos == B Y hab_A_sucia
ENTONCES mover_izquierda
```

© gonzalezhalexis@gmail.com ©

Programa de los Agentes

Ejemplo 3: Agente con Modelo del Mundo • Preguntas para Reflexionar

1. Importancia del Modelo

¿Por qué el modelo es importante?

Permite razonar sobre el mundo sin percibirlo directamente.

2. Impacto de Errores

¿Qué pasa si el modelo es incorrecto?

El agente toma decisiones subóptimas o incorrectas.

3. Dinámica del Entorno

¿Cómo actualizar si el entorno cambia?

Necesitamos **actualización continua** del modelo.

4. Evaluación de Racionalidad

¿Este agente es racional?

Más racional que el simple, pero aún subóptimo.

5. Puntos de Falla

¿Cuándo falla este agente?

Entornos muy dinámicos o estocásticos.

© gonzalezhalexis@gmail.com ©

Resolución de problemas mediante búsqueda

Agentes de resolución de problemas

ENTORNO Y MARCO DEL AGENTE (Condiciones del Tablero)

Agente de Resolución:
Planifica con anticipación una secuencia de acciones hacia un estado objetivo mediante el proceso de **búsqueda**.

Condiciones del Entorno Considerado:
Episódico y Agente único Totalmente observable Determinista y Estático Discreto y Conocido

1. Objetivos



Adopción de metas

El agente adopta el objetivo basado en la situación actual y en su medida de rendimiento.

2. Problema



Modelo abstracto

Diseña una descripción de estados y acciones necesarias para alcanzar el objetivo en el entorno.

3. Búsqueda



Simulación previa

Simula secuencias de acciones en su modelo abstracto hasta hallar una secuencia solución válida.

4. Ejecución



Acción en el entorno

Lleva a cabo las acciones de la solución obtenida en el mundo real, un paso a la vez.

© gonzalezhalexis@gmail.com ©

Resolución de problemas mediante búsqueda

Problemas de búsquedas y soluciones • Definición Formal

1. Espacio de Estados

Entorno Posible

Conjunto de todos los estados posibles en los que puede estar el entorno durante el proceso.



2. Estado Inicial y Objetivos

- **Estado Inicial:**

Estado en el que el agente comienza la búsqueda.

- **Estados Objetivos:**

Set de uno o más estados meta a alcanzar.

3. Acciones Disponibles

ACTIONS(s)

Dado un estado **s**, retorna un número finito de acciones aplicables que el agente puede ejecutar.

4. Modelo de Transición

RESULT(s, a)

Describe el comportamiento de cada acción devolviendo el estado resultante tras realizar la acción **a** en **s**.

5. Función de Costo

ACTION-COST(s, a, s')

Asigna un valor numérico al costo de aplicar la acción **a** en el estado **s** para llegar a **s'**.

6. Solución y Optimalidad

- **Secuencia de Acciones:**

Forma un camino desde el estado inicial al objetivo.

- **Solución Óptima:**

Aquella que logra el menor costo total acumulado.

© gonzalezhalexis@gmail.com ©

Torre de Hanoi

Origen y Leyenda

Origen Histórico

Inventado en 1883 por el matemático francés Édouard Lucas.

La Leyenda de Brahma

En un templo de Benarés, unos brahmanes mueven 64 discos de oro sin parar desde hace siglos.

⚠️ **El Fin del Mundo**

Una vez finalizada la torre, llegará el fin del mundo.

Representación Visual



Objetivo del Juego

Mover toda la pila de discos a una de las otras varillas manteniendo el orden cónico inicial.

Reglas del Juego

1. Movimiento Individual

Sólo se puede mover un disco a la vez.

2. Destino Permitido

Cada movimiento consiste en tomar el disco superior de una pila y colocarlo en otra pila o varilla vacía.

3. Restricción de Tamaño

Ningún disco podrá colocarse encima de uno más pequeño que él.

- **Las Acciones Disponibles**
  ACTIONS(s)
- **El Modelo de Transición**
  RESULT(s, a)
- **La Función de Costo**
  ACTION-COST(s, a, s')

© gonzalezhalexis@gmail.com ©

Torre de Hanoi

Resolviendo este problema usando IA • Enfoque de Búsqueda

Planteo del Problema y Agente

Formulación para Búsqueda

Este es un típico problema para aplicar métodos de búsqueda. Podemos crear un agente capaz de encontrar la secuencia óptima de solución.

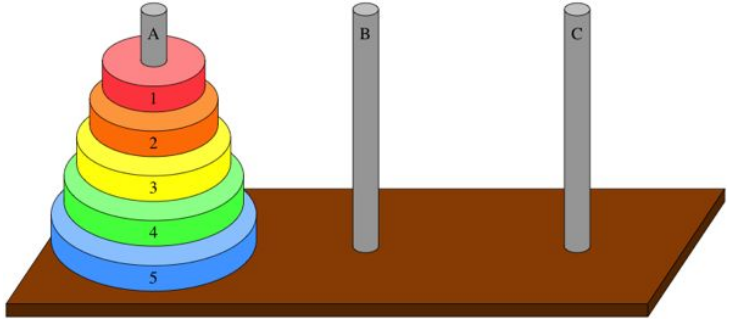
Dominio y Percepción

- **Tamaño del Problema:** Limitemos a 5 discos (a diferencia de los 64 discos de la leyenda).
- **Sensores / Percepción:** El agente percibe cuántos discos y en qué orden se encuentran en cada varilla.

Acciones Validadas

- **Manipulación:** El agente puede tomar cualquier disco situado en la parte superior.
- **Transición:** Mover el disco a cualquier otra varilla donde la regla lo permita.

Representación del Estado Inicial



© gonzalezhalexis@gmail.com ©

Torre de Hanoi

Resolviendo este problema usando IA

Formulación del Problema

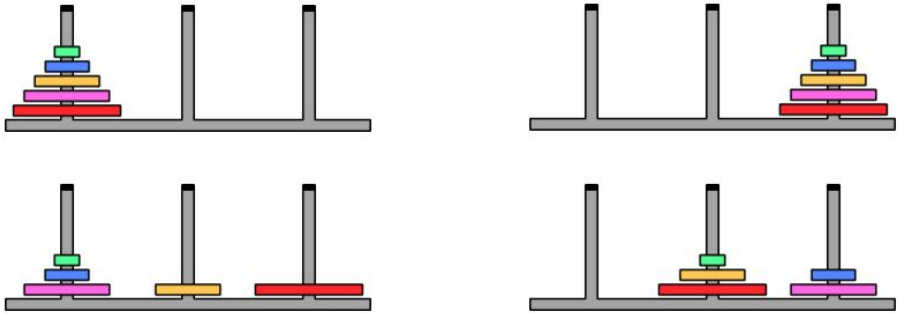
Definamos el problema con las características que vimos:

Espacio de Estados

243

Para 5 discos, tenemos 3^5 = 243 posibles estados

Representación de Estados



© gonzalezhalexis@gmail.com ©

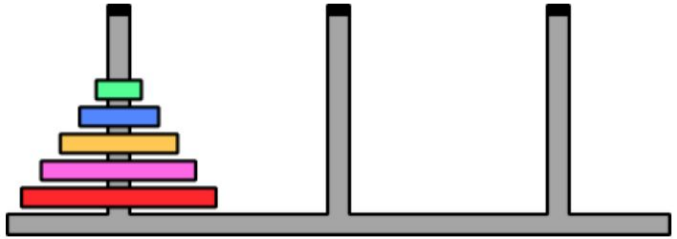
Torre de Hanoi

Resolviendo este problema usando IA

Definamos el problema con las características principales de sus estados:

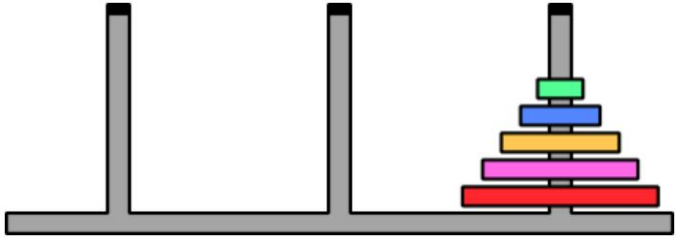
Estado Inicial

Todos los discos se encuentran ordenados por tamaño en la primera varilla (origen).



Estado Objetivo

Para simplificar, vamos a tener un solo estado objetivo de los dos posibles.



© gonzalezhalexis@gmail.com ©

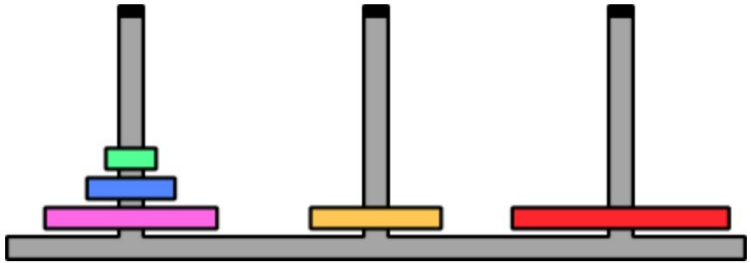
Torre de Hanoi

Resolviendo este problema usando IA

Definamos el problema con las características que vimos:

Estado de Ejemplo (s)

Acciones ACTIONS(s), por ejemplo, para el siguiente estado, tenemos las siguientes acciones:



Acciones Posibles: ACTIONS(s)

Movimientos válidos desde la configuración actual:

Disco	Acción
--	No hacer nada
Amarillo	Mover a la varilla derecha
Verde	Mover a la varilla del medio
Verde	Mover a la varilla derecha

© gonzalezhalexis@gmail.com ©

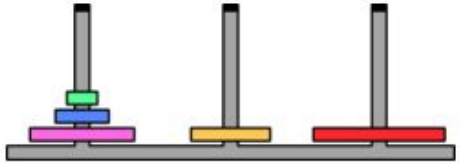
Torre de Hanoi

Resolviendo este problema usando IA

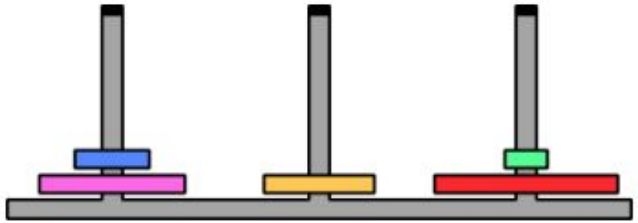
Definamos el problema con las características que vimos:

Modelo de transición: RESULT(s,a)

Por ejemplo:



Disco	Acción
--	No hacer nada
Amarillo	Mover a la varilla derecha
Verde	Mover a la varilla del medio
Verde	Mover a la varilla derecha



© gonzalezhalexis@gmail.com ©

Torre de Hanoi

Resolviendo este problema usando IA

Definamos el problema con las características que vimos:

Función de costo de acción

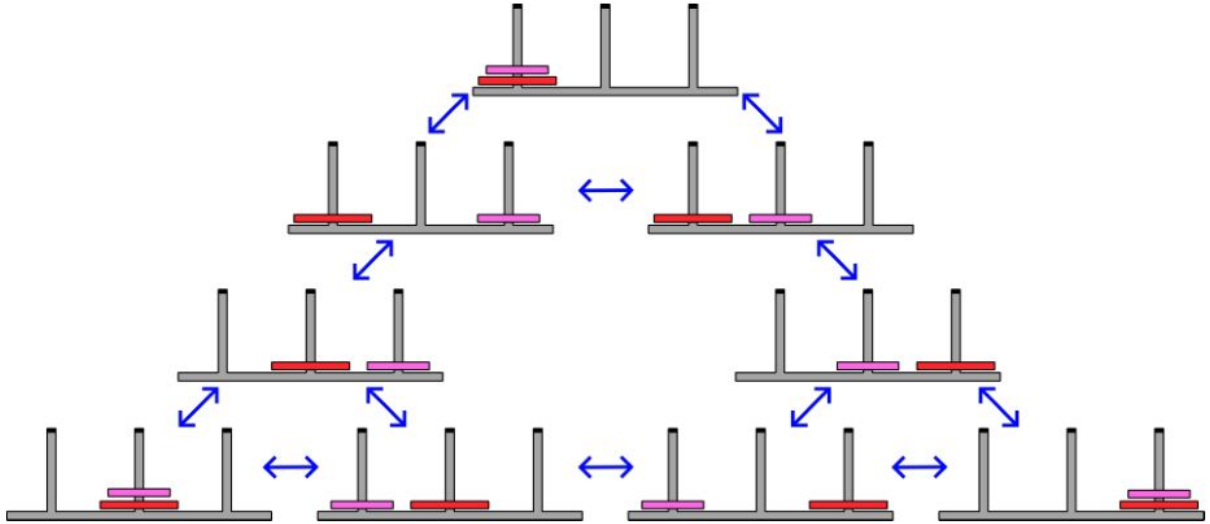
ACTION-COST(s, a, s')

Mover un disco de una varilla a otra, siempre que sea un movimiento permitido, cuesta lo mismo para cualquier estado.

Valor del costo

c = 1

Grafo de estados



© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda

Árbol de búsqueda

Concepto Principal

Un algoritmo de búsqueda toma un problema como entrada y retorna una solución o una indicación de falla.

Superpone un **árbol de búsqueda** sobre el grafo de estados para encontrar un camino al estado objetivo.

Componentes del Árbol

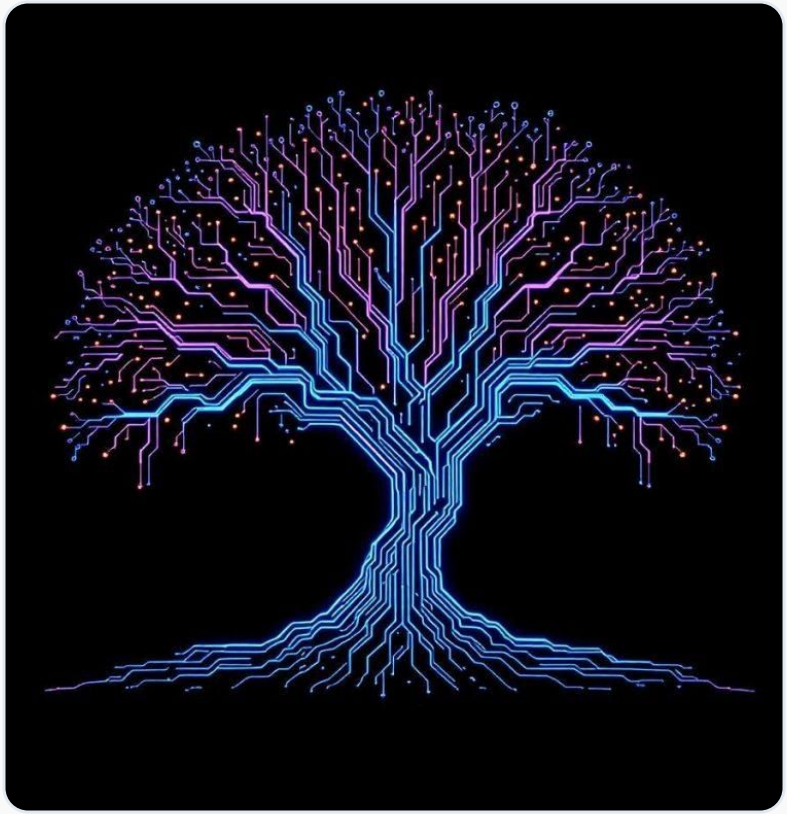
- **Nodo:** Corresponde a un **estado**.
- **Arista:** Corresponde a una **acción**.

Diferencia Clave: Árbol vs. Grafo

El árbol NO es el grafo de estados:

Grafo de estados: Describe *todo* el conjunto de estados y las acciones que llevan de un lado a otro.

Árbol de búsqueda: Describe el *camino específico* entre estos estados para alcanzar el objetivo.



© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda

Árbol de búsqueda

Estado Raíz (Inicio)

Punto de partida del algoritmo (recuadro superior). Muestra la configuración inicial del problema.

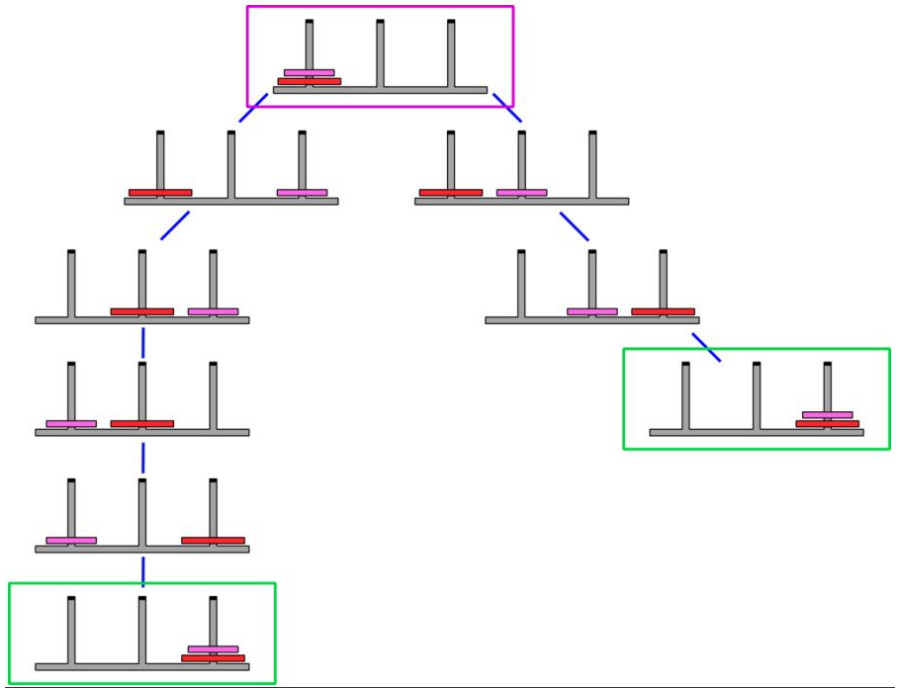
Expansión y Ramas

Cada nivel genera nuevos estados válidos aplicando las acciones o movimientos permitidos.

Estados Objetivo (Solución)

Las hojas finales (destacadas en verde) representan los estados meta exitosos alcanzados.

Estructura del Árbol de Búsqueda (Torre de Hanoi)



© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda

Árbol de búsqueda

Frontera de Búsqueda

La frontera separa dos regiones del grafo, aquella que ya fue explorada por el algoritmo y aquella que no.

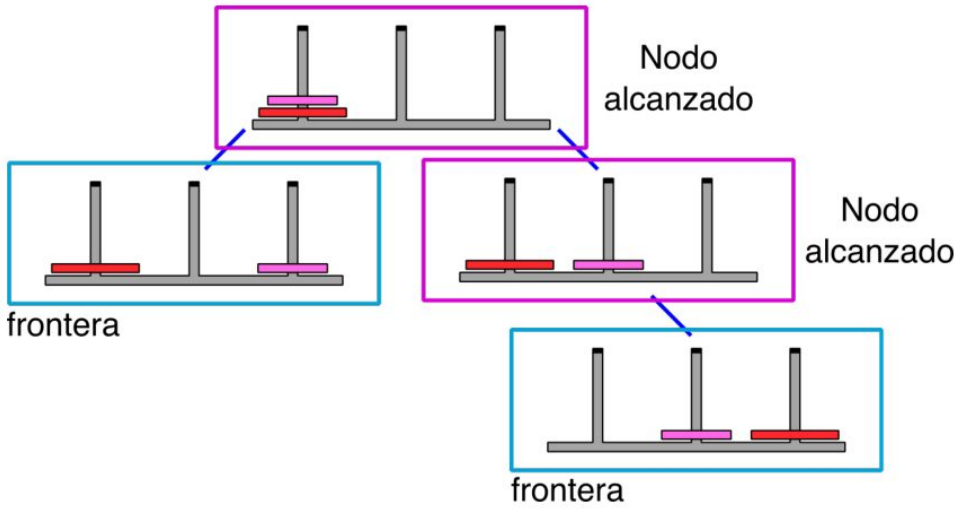
Región Explorada

Nodos que ya han sido visitados y evaluados por el algoritmo. Se encuentran en el espacio interior delimitado por la frontera.

Región No Explorada

Estados desconocidos que aún no han sido alcanzados ni procesados en el árbol de búsqueda.

Visualización de la Frontera y Nodos Alcanzados



© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda

Árbol de búsqueda – Estructura de datos

Para poder aplicar los algoritmos, debemos definir la estructura de datos para hacer seguimiento del árbol. Los nodos del árbol son representados con los siguientes componentes:

STATE



El estado, del espacio de estados, que corresponde el nodo.

NODE PARENT



El nodo en el árbol de búsqueda que ha generado al nodo.

ACTION



La acción que se aplicará al padre para generar el nodo.

PATH-COST



El costo g(n) de un camino desde el nodo inicial al nodo.

© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda

Árbol de búsqueda – Estructura de datos

Necesitamos una estructura para la frontera. Seleccionamos una cola, porque las operaciones en la frontera son:

IS-EMPTY



IS-EMPTY(FRONTIER)

Retorna True si no hay nodos en la frontera.

POP



POP(FRONTIER)

Quita el primer nodo en la cola.

TOP



TOP(FRONTIER)

Devuelve, pero no quita al primer nodo en la cola.

ADD



ADD(FRONTIER)

Inserta el nodo en su correspondiente lugar de la cola.

© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda

Árbol de búsqueda – Estructura de datos

Tres tipos de colas se utilizan en los algoritmos de búsqueda, determinando el orden de exploración de los nodos:

1. Cola FIFO (First-In, First-Out)

Toma los nodos en el mismo orden en que se agregan a la frontera.

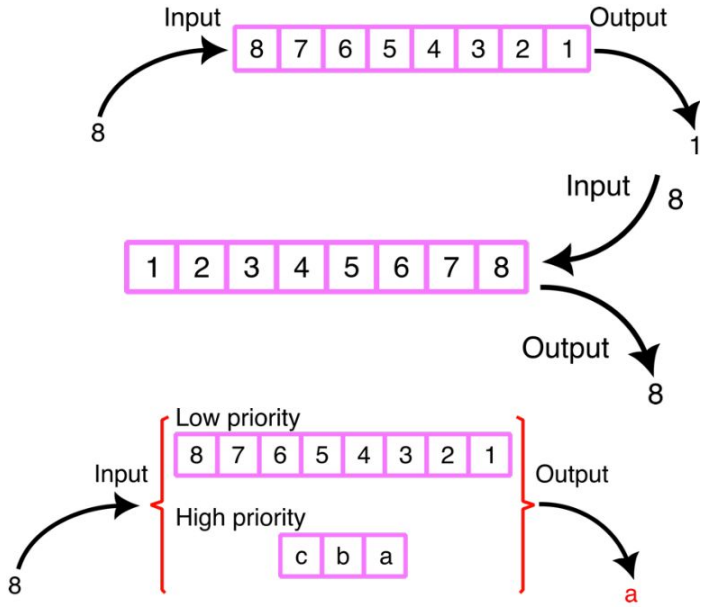
2. Cola LIFO (Last-In, First-Out / Stack)

Quita el nodo más reciente introducido en la frontera.

3. Cola Prioritaria (Priority Queue)

Quita primero los nodos con el mínimo costo según una función de evaluación *f*.

Representación Gráfica de los Comportamientos



© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda

Midiendo el rendimiento

Para poder evaluar a los algoritmos de búsquedas, debemos usar un criterio para elegir:

Completitud

¿El algoritmo garantiza encontrar una solución cuando hay una, y correctamente informar cuando no lo haya?

Optimización

¿Encuentra la estrategia la solución óptima, es decir el camino más corto?

Complejidad de Tiempo

¿Cuánto tiempo le lleva encontrar la solución?

Complejidad de Espacio

¿Cuánta memoria es necesaria para la búsqueda?

© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda

Clasificación de los algoritmos a estudiar

Hay múltiples algoritmos de búsqueda, los que veremos hoy son:

Algoritmos de búsqueda no informada

- Búsqueda primero en anchura
- Búsqueda de costo uniforme o algoritmo de Dijkstra
- Búsqueda primero en profundidad
- Búsqueda de profundidad limitada
- Búsqueda de profundidad limitada con profundidad iterativa

Algoritmos de búsqueda informada

- Búsqueda voraz (greedy) primero el mejor
- Búsqueda A*

© gonzalezhalexis@gmail.com ©

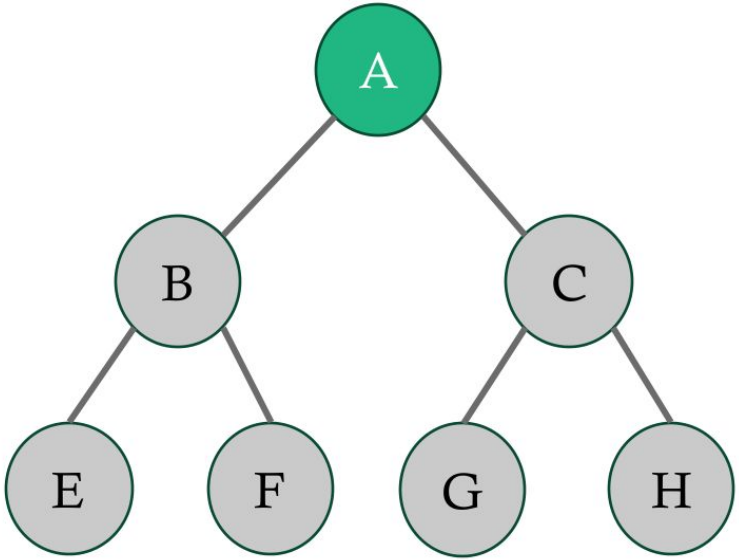
Algoritmos de Búsqueda No Informada

Búsqueda primero en anchura (BFS)

Estrategia en la que se expande primero el nodo raíz y, a continuación, se expanden todos los sucesores nivel por nivel.

Complejidad Temporal / Espacial: O(b^(d+1))

Árbol de Búsqueda



Estructura de Datos: Cola FIFO

First In, First Out (Primero en entrar, primero en salir)



- Los nodos descubiertos se añaden al final de la cola.
- Se extrae el nodo frontal para su expansión.
- Garantiza encontrar la solución con menor profundidad en grafos no ponderados.

© gonzalezhalexis@gmail.com ©

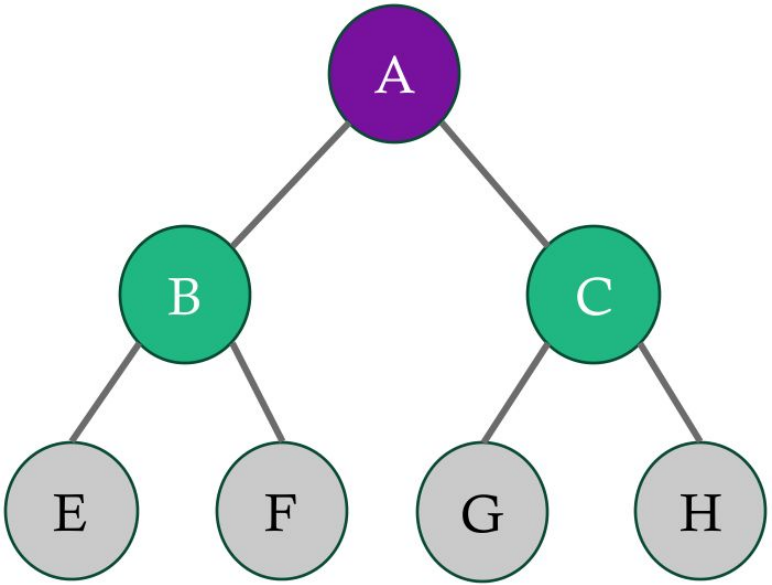
Algoritmos de Búsqueda No Informada

Búsqueda primero en anchura (BFS)

Estrategia en la que se expande primero el nodo raíz y, a continuación, se expanden todos los sucesores nivel por nivel.

Complejidad Temporal / Espacial: O(b^d+1)

Árbol de Búsqueda



Estructura de Datos: Cola FIFO

First In, First Out (Primero en entrar, primero en salir)



© gonzalezhalexis@gmail.com ©

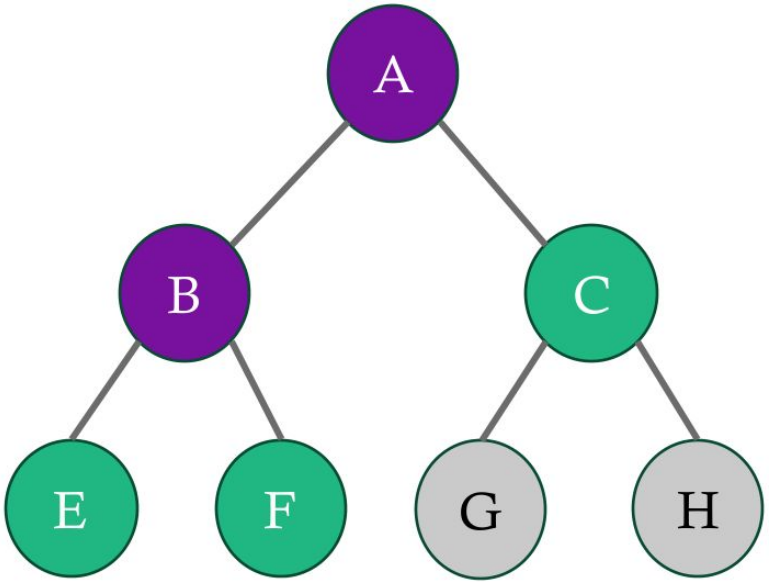
Algoritmos de Búsqueda No Informada

Búsqueda primero en anchura (BFS)

Estrategia en la que se expande primero el nodo raíz y, a continuación, se expanden todos los sucesores nivel por nivel.

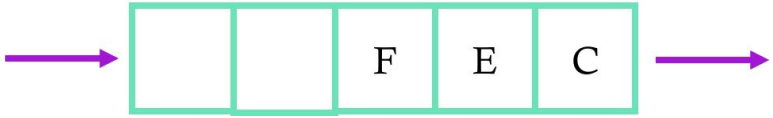
Complejidad Temporal / Espacial: O(b^d+1)

Árbol de Búsqueda



Estructura de Datos: Cola FIFO

First In, First Out (Primero en entrar, primero en salir)



© gonzalezhalexis@gmail.com ©

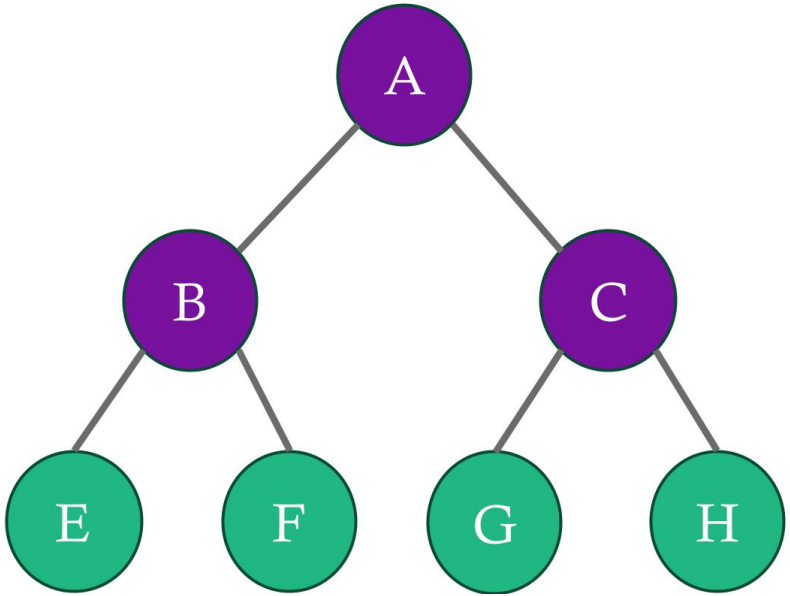
Algoritmos de Búsqueda No Informada

Búsqueda primero en anchura (BFS)

Estrategia en la que se expande primero el nodo raíz y, a continuación, se expanden todos los sucesores nivel por nivel.

Complejidad Temporal / Espacial: O(b^d+1)

Árbol de Búsqueda



Estructura de Datos: Cola FIFO

First In, First Out (Primero en entrar, primero en salir)



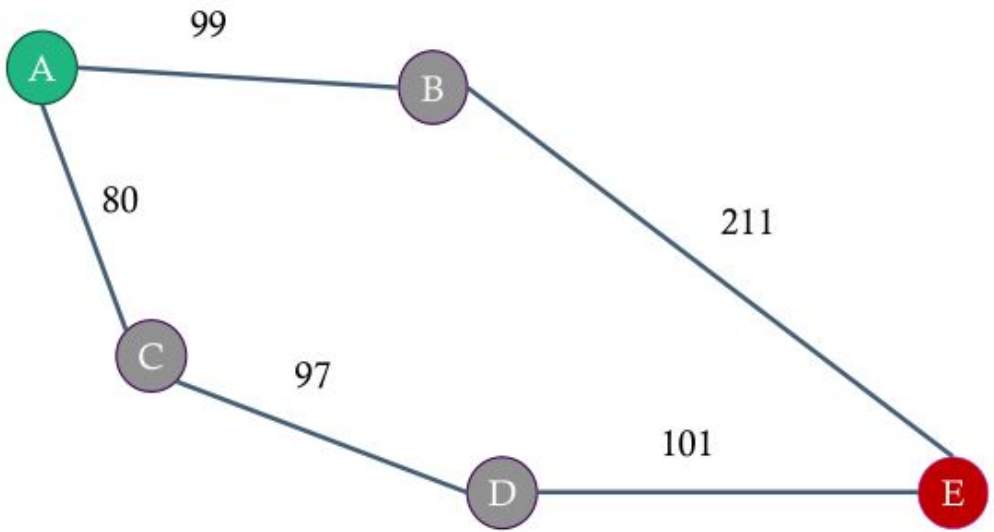
© gonzalezhalexis@gmail.com ©

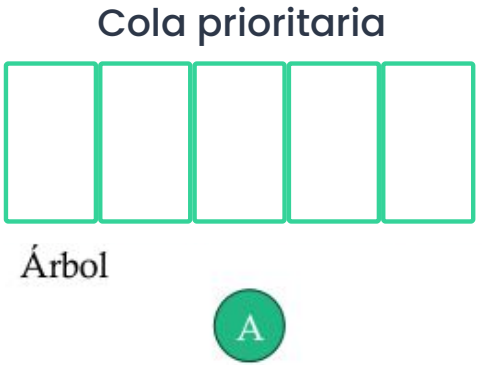
Algoritmos de Búsqueda No Informada | Búsqueda de Costo Uniforme (Dijkstra)

Estrategia que expande el nodo *n* con el menor costo de camino acumulado *g(n)*.

Estructura de Datos: Cola de Prioridad (ordenada por g(n))

Grafo de Estados y Cola de Prioridad





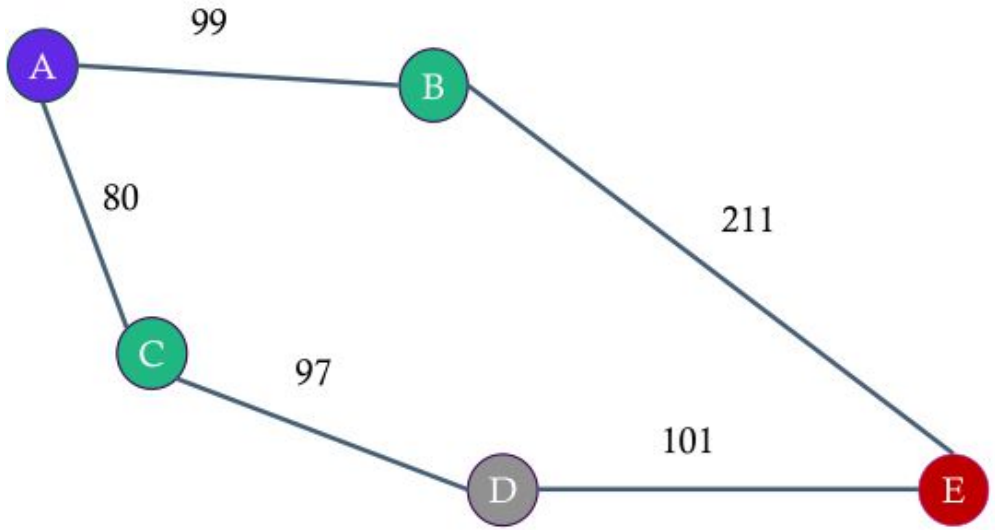
© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda No Informada | Búsqueda de Costo Uniforme (Dijkstra)

Estrategia que expande el nodo *n* con el menor costo de camino acumulado *g(n)*.

Estructura de Datos: Cola de Prioridad (ordenada por g(n))

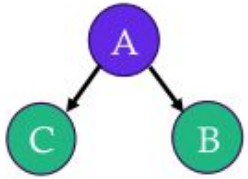
Grafo de Estados y Cola de Prioridad



Cola prioritaria



Árbol



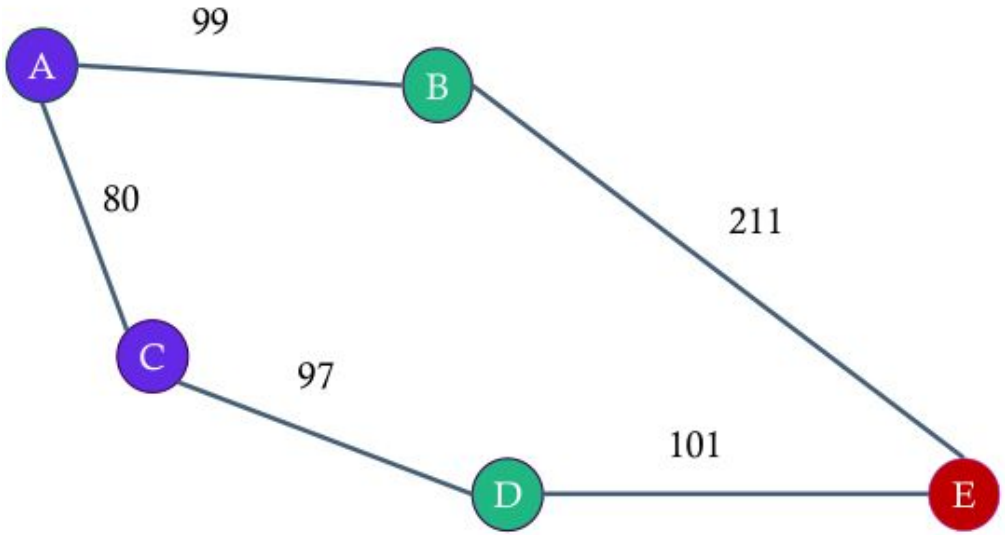
© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda No Informada | Búsqueda de Costo Uniforme (Dijkstra)

Estrategia que expande el nodo *n* con el menor costo de camino acumulado *g(n)*.

Estructura de Datos: Cola de Prioridad (ordenada por g(n))

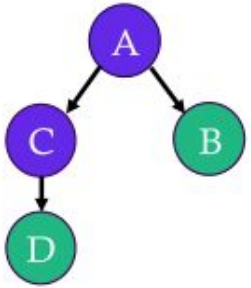
Grafo de Estados y Cola de Prioridad



Cola prioritaria



Arbol



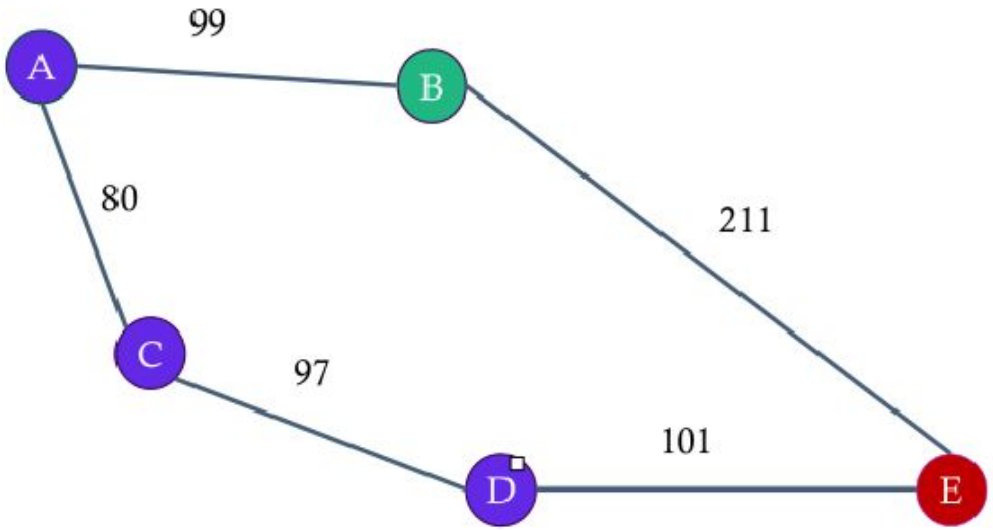
© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda No Informada | Búsqueda de Costo Uniforme (Dijkstra)

Estrategia que expande el nodo *n* con el menor costo de camino acumulado *g(n)*.

Estructura de Datos: Cola de Prioridad (ordenada por g(n))

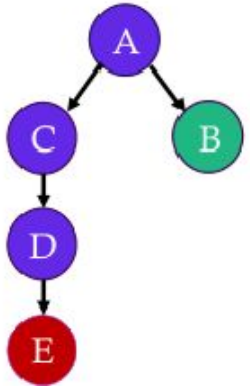
Grafo de Estados y Cola de Prioridad



Cola prioritaria

			D	E
			177	310

Árbol



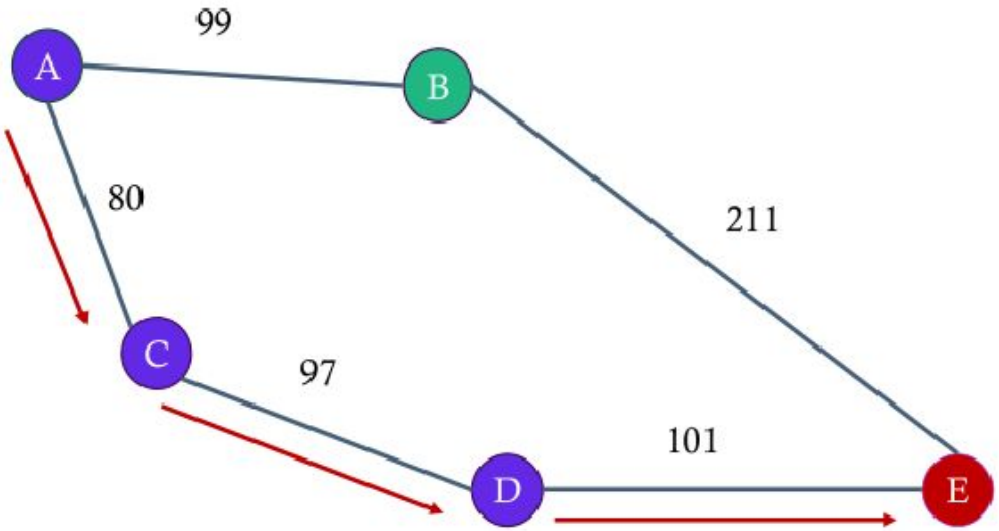
© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda No Informada | Búsqueda de Costo Uniforme (Dijkstra)

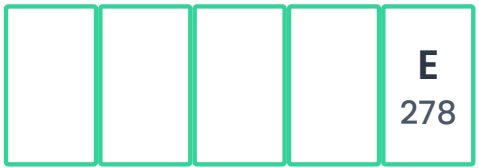
Estrategia que expande el nodo *n* con el menor costo de camino acumulado *g(n)*.

Estructura de Datos: Cola de Prioridad (ordenada por g(n))

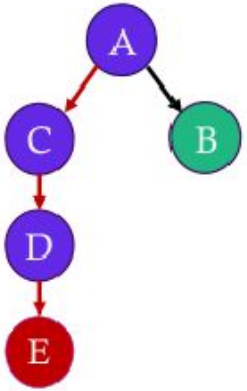
Grafo de Estados y Cola de Prioridad



Cola prioritaria



Arbol



© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda No Informada | Búsqueda Primero en Profundidad (DFS)

Estrategia que expande siempre el nodo más profundo en la frontera actual del árbol de búsqueda.

Estructura de Datos: Pila / LIFO (Last-In, First-Out)

Mecánica de Búsqueda

Siempre expande el nodo más profundo en la frontera actual del árbol de búsqueda.

Cuando esos nodos se expanden, son quitados de la frontera, así la búsqueda **"retrocede"** al siguiente nodo más superficial.

Uso de Memoria

O(bm)

Consume **muy poca memoria** en comparación con otros métodos no informados.

Tiempo y Eficiencia

No encuentra la solución más eficiente (no es óptimo).

El tiempo de ejecución es directamente **proporcional a la cantidad de estados**.

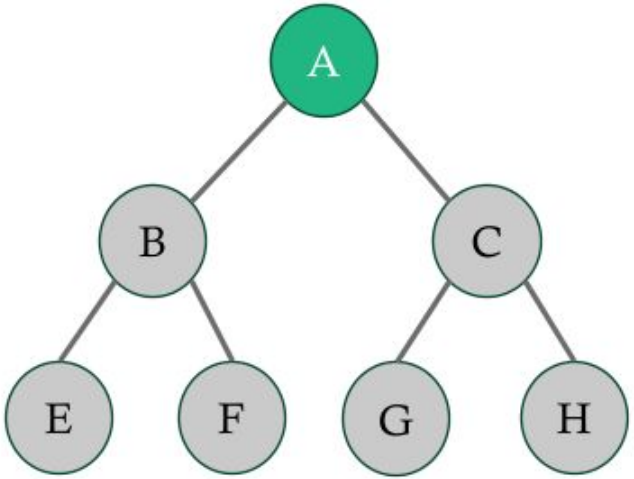
© gonzalezhalexis@gmail.com ©

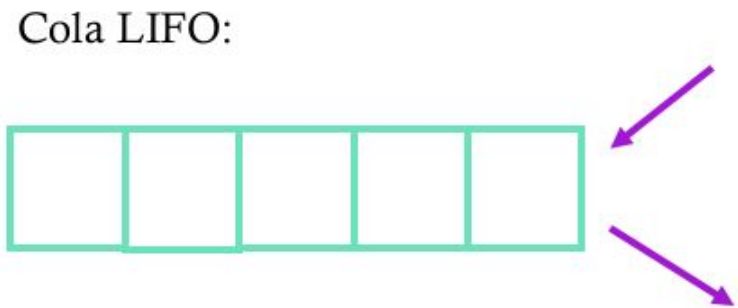
Algoritmos de Búsqueda No Informada | Búsqueda Primero en Profundidad (DFS)

Estrategia que expande siempre el nodo más profundo en la frontera actual del árbol de búsqueda.

Estructura de Datos: Pila / LIFO (Last-In, First-Out)

Árbol de Búsqueda y Pila LIFO





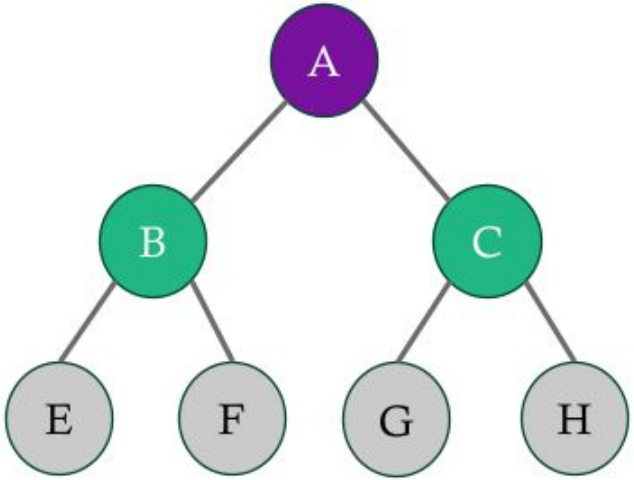
© gonzalezhalexis@gmail.com ©

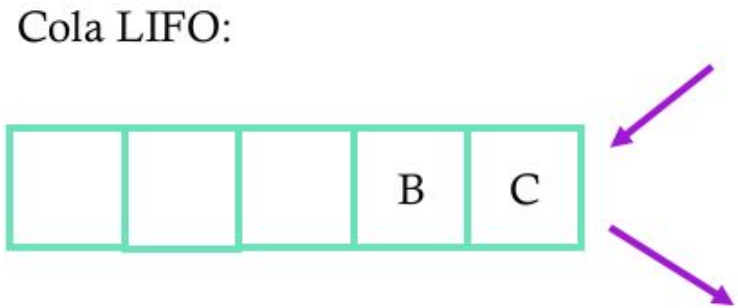
Algoritmos de Búsqueda No Informada | Búsqueda Primero en Profundidad (DFS)

Estrategia que expande siempre el nodo más profundo en la frontera actual del árbol de búsqueda.

Estructura de Datos: Pila / LIFO (Last-In, First-Out)

Árbol de Búsqueda y Pila LIFO





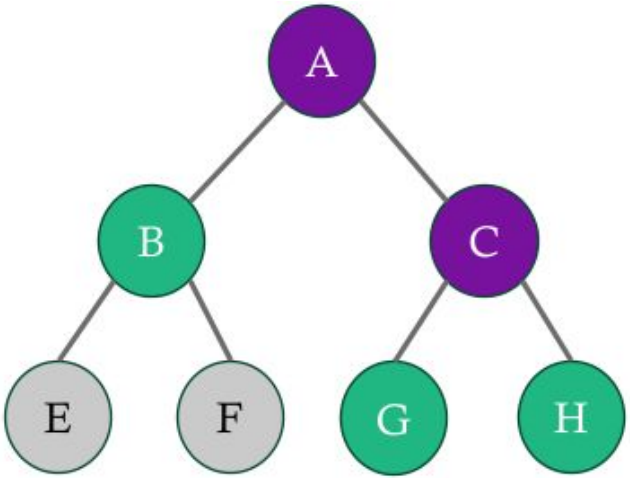
© gonzalezhalexis@gmail.com ©

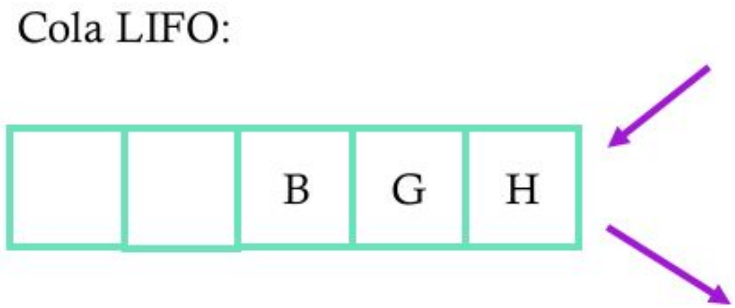
Algoritmos de Búsqueda No Informada | Búsqueda Primero en Profundidad (DFS)

Estrategia que expande siempre el nodo más profundo en la frontera actual del árbol de búsqueda.

Estructura de Datos: Pila / LIFO (Last-In, First-Out)

Árbol de Búsqueda y Pila LIFO





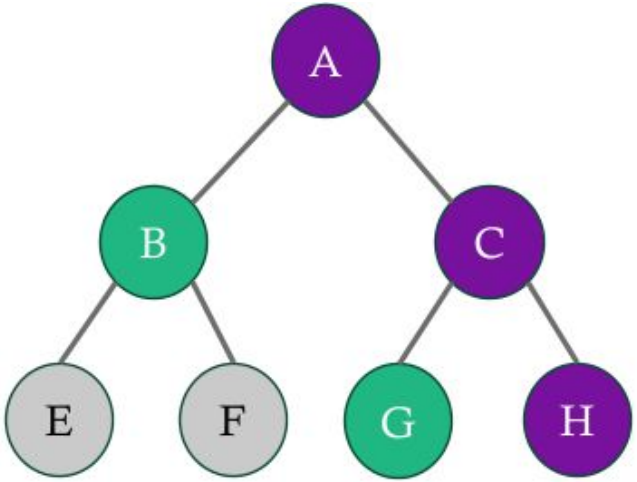
© gonzalezhalexis@gmail.com ©

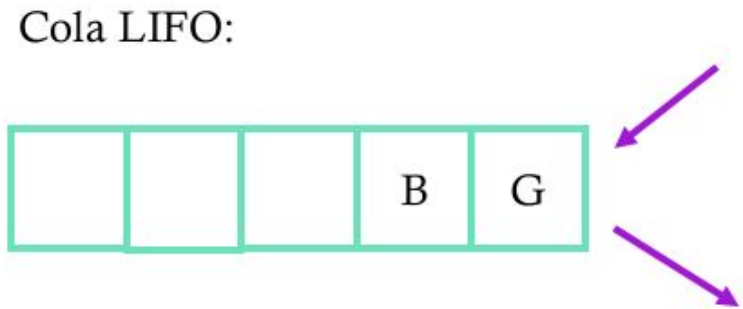
Algoritmos de Búsqueda No Informada | Búsqueda Primero en Profundidad (DFS)

Estrategia que expande siempre el nodo más profundo en la frontera actual del árbol de búsqueda.

Estructura de Datos: Pila / LIFO (Last-In, First-Out)

Árbol de Búsqueda y Pila LIFO





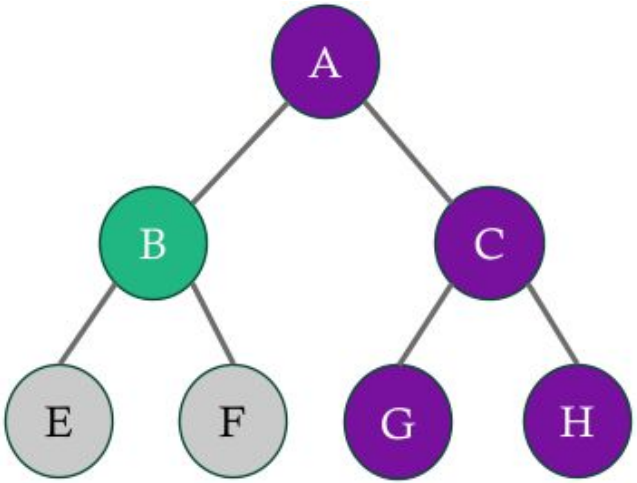
© gonzalezhalexis@gmail.com ©

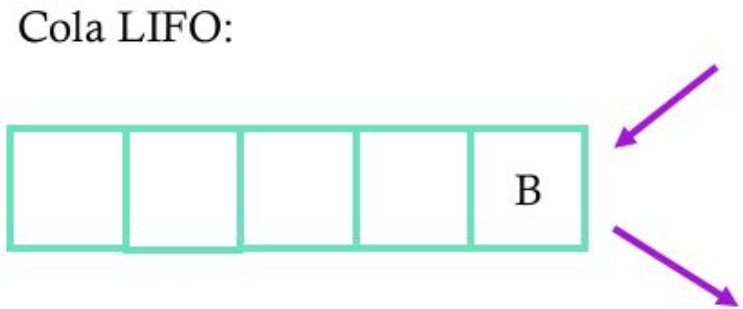
Algoritmos de Búsqueda No Informada | Búsqueda Primero en Profundidad (DFS)

Estrategia que expande siempre el nodo más profundo en la frontera actual del árbol de búsqueda.

Estructura de Datos: Pila / LIFO (Last-In, First-Out)

Árbol de Búsqueda y Pila LIFO





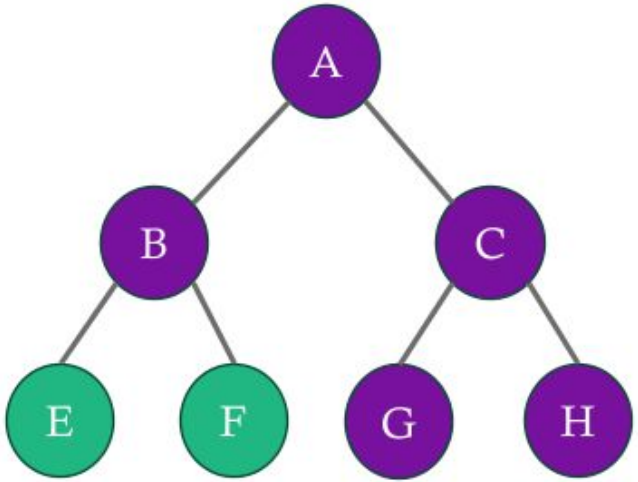
© gonzalezhalexis@gmail.com ©

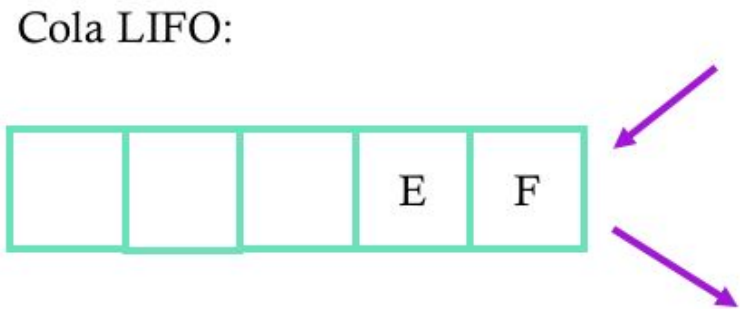
Algoritmos de Búsqueda No Informada | Búsqueda Primero en Profundidad (DFS)

Estrategia que expande siempre el nodo más profundo en la frontera actual del árbol de búsqueda.

Estructura de Datos: Pila / LIFO (Last-In, First-Out)

Árbol de Búsqueda y Pila LIFO





© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda No Informada | Búsqueda de Profundidad Limitada

Estrategia que acota la exploración en profundidad hasta un nivel predeterminado para evitar ramas infinitas.

Parámetro Principal: Límite de Profundidad (l)

Funcionamiento General

- **Casos de Aplicación:** Para árboles de búsqueda muy grandes o infinitos, se limita la exploración hasta un cierto nivel l.
- **Ejemplo de Nivel:** Si elegimos una profundidad de 2, la búsqueda solo llegará hasta evaluar los nodos B y C.
- **Restricción de Avance:** Restringe qué tan profundo avanza el algoritmo en el árbol.
- **Desafío Clave:** La principal dificultad radica en saber elegir la profundidad l adecuada.

Complejidad en Tiempo

$O(b^l)$

b: factor de ramificación | l: límite de profundidad

Complejidad en Memoria

$O(b \cdot l)$

Consumo de espacio lineal proporcional al límite

Limitación Importante

Si la solución se encuentra a una profundidad mayor a **l**, el algoritmo no la encontrará, comprometiendo la completitud.

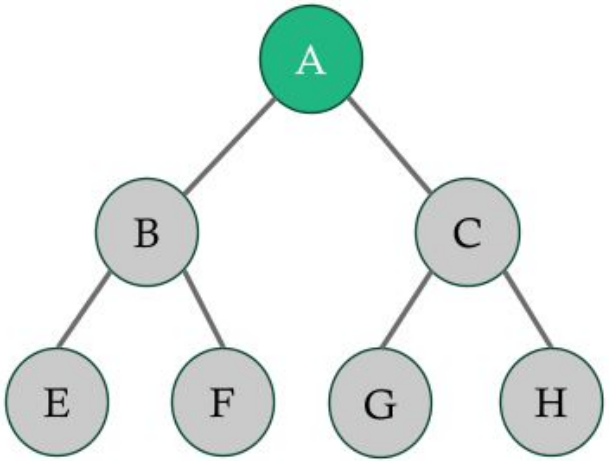
© gonzalezhalexis@gmail.com ©

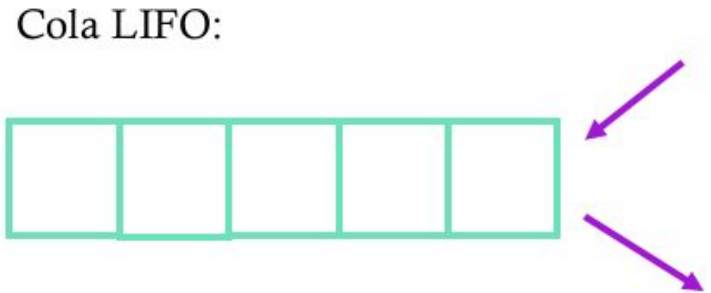
Algoritmos de Búsqueda No Informada | Búsqueda de Profundidad Limitada

Estrategia que acota la exploración en profundidad hasta un nivel predeterminado para evitar ramas infinitas.

Parámetro Principal: Límite de Profundidad (l)

Búsqueda de profundidad limitada (Límite 2)





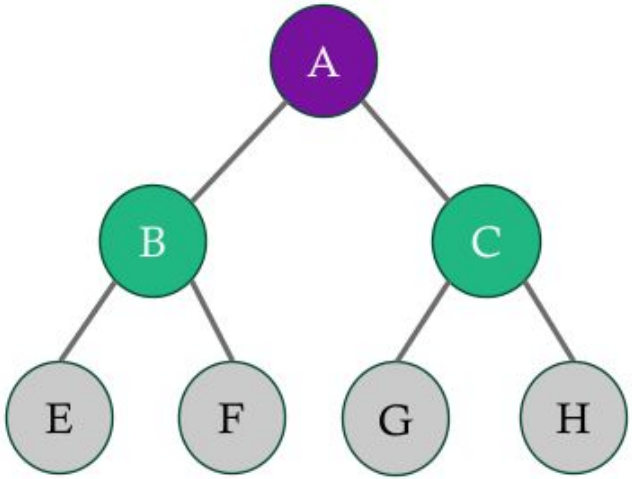
© gonzalezhalexis@gmail.com ©

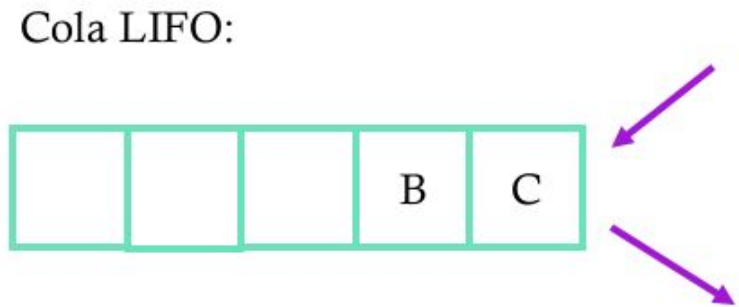
Algoritmos de Búsqueda No Informada | Búsqueda de Profundidad Limitada

Estrategia que acota la exploración en profundidad hasta un nivel predeterminado para evitar ramas infinitas.

Parámetro Principal: Límite de Profundidad (l)

Búsqueda de profundidad limitada (Límite 2)





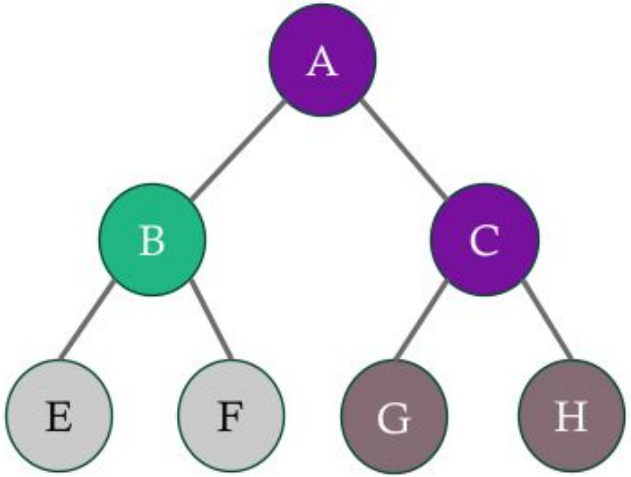
© gonzalezhalexis@gmail.com ©

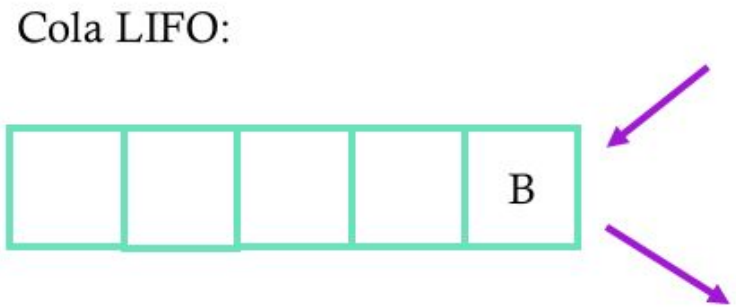
Algoritmos de Búsqueda No Informada | Búsqueda de Profundidad Limitada

Estrategia que acota la exploración en profundidad hasta un nivel predeterminado para evitar ramas infinitas.

Parámetro Principal: Límite de Profundidad (l)

Búsqueda de profundidad limitada (Límite 2)





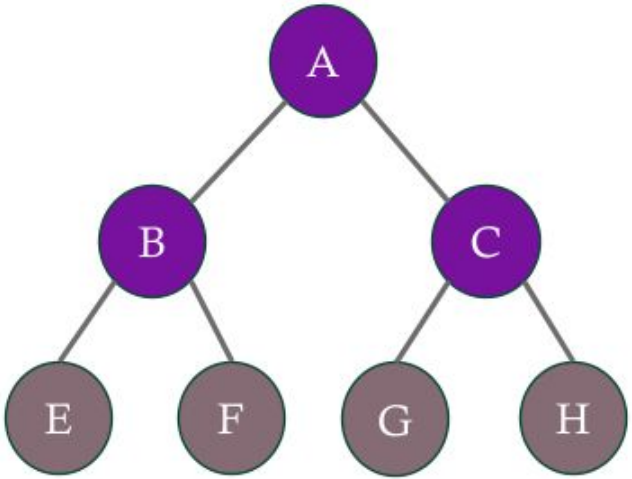
© gonzalezhalexis@gmail.com ©

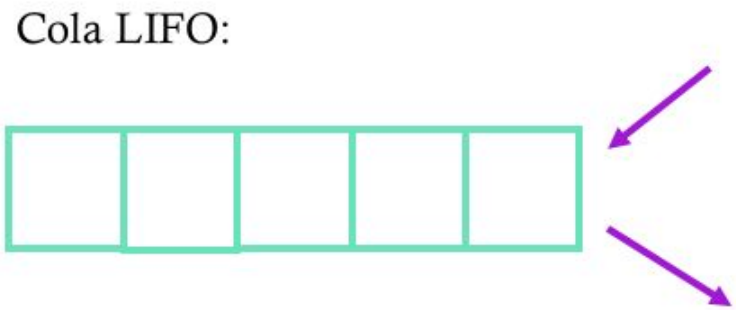
Algoritmos de Búsqueda No Informada | Búsqueda de Profundidad Limitada

Estrategia que acota la exploración en profundidad hasta un nivel predeterminado para evitar ramas infinitas.

Parámetro Principal: Límite de Profundidad (l)

Búsqueda de profundidad limitada (Límite 2)





© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda No Informada | Búsqueda de Profundidad Iterativa

Estrategia que explora el árbol incrementando el límite de profundidad en cada paso hasta encontrar la solución.

Secuencia de Límite: l = 0, 1, 2, 3, ...

¿Cómo Funciona?

Esta búsqueda es una estrategia que se realiza aumentando la profundidad iterativamente hasta alcanzar la solución.

Se parte de un límite de **0, 1, 2, ...** ejecutando una Búsqueda de Profundidad Limitada en cada iteración.

Característica Clave:

Combina la eficiencia en memoria de DFS con la completitud de BFS.

Uso Recomendado

Es el método preferido de búsqueda no informada cuando:

- Existe un **espacio grande de búsqueda**.
- **No se conoce** la profundidad de la solución de antemano.

Ventaja Principal:

Garantiza encontrar la solución más cercana a la raíz sin quedar atrapado en ramas infinitas.

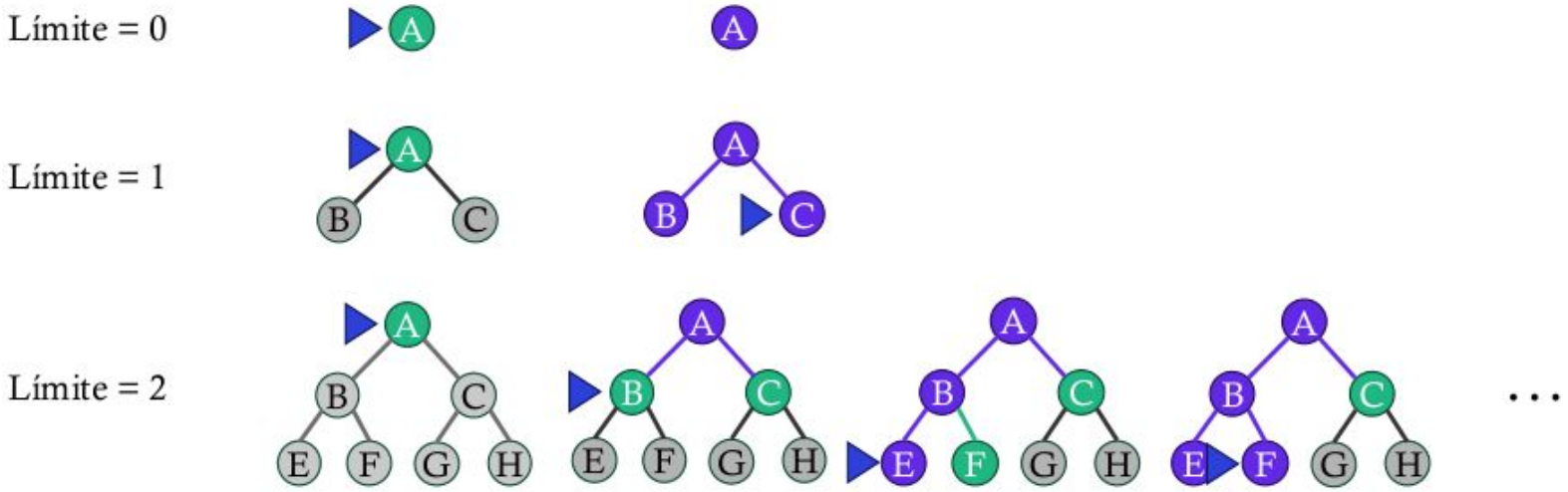
© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda No Informada | Búsqueda de Profundidad Iterativa

Estrategia que explora el árbol incrementando el límite de profundidad en cada paso hasta encontrar la solución.

Secuencia de Límite: l = 0, 1, 2, 3, ...

Visualización del Proceso por Niveles de Límite



© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda Informada | Función Heurística

El Desafío de la Búsqueda

Saber qué tan lejos estamos del objetivo permitiría realizar una búsqueda mucho más eficiente que probar caminos a ciegas.

Problema: Conocer la distancia exacta requeriría haber resuelto el problema primero.

La Solución: Estimación Heurística

Resolvemos esto mediante una estimación aproximada de la distancia hacia la meta:

h(n)

Costo estimado del camino más barato desde el estado del nodo **n** al estado objetivo.



© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda Informada | Búsqueda Voraz (Greedy) Primero el Mejor

Estrategia de Expansión

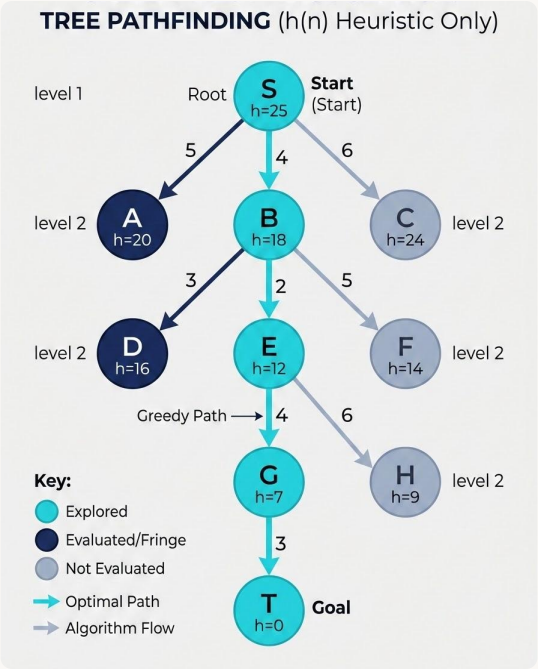
Expande el nodo con el valor más bajo de **h(n)** (el nodo que parece más cercano al objetivo).

Premisa: Evaluar la opción localmente óptima para conducir rápidamente a la solución.

Propiedades del Algoritmo

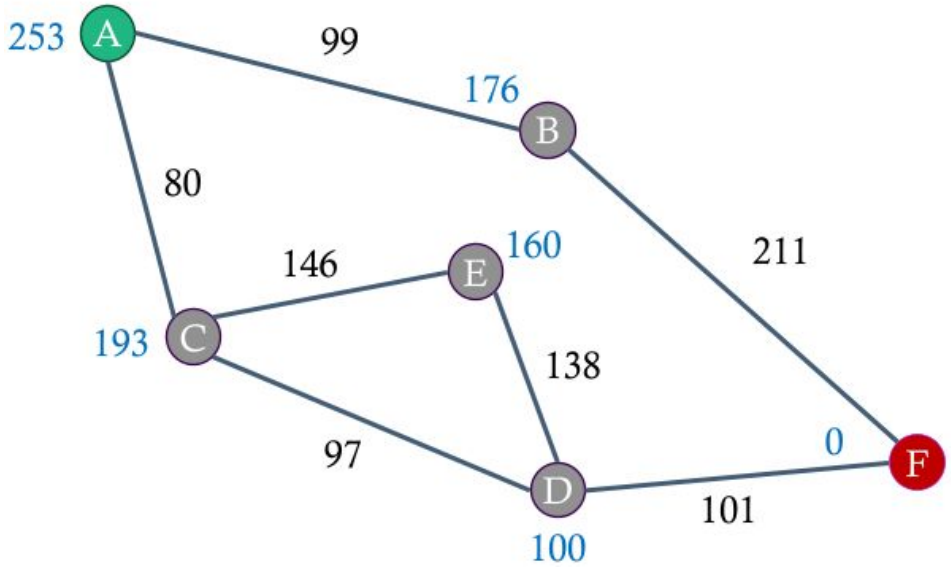
⚡ **Mayor Rapidez:** Permite alcanzar la solución más rápido que los métodos de búsqueda no informada.

⚠️ **No Garantiza Optimalidad:** Esta estrategia no siempre asegura encontrar el camino con el costo mínimo.

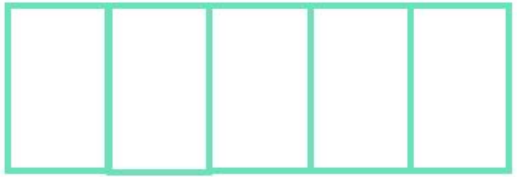


© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda Informada | Búsqueda Voraz (Greedy) Primero el Mejor



Cola prioritaria

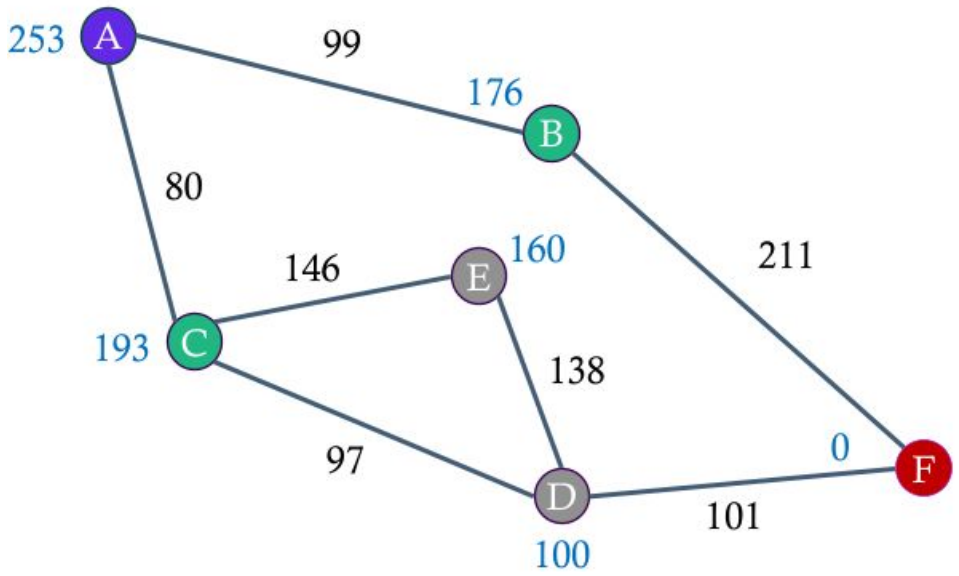


Árbol



© gonzalezhalexis@gmail.com ©

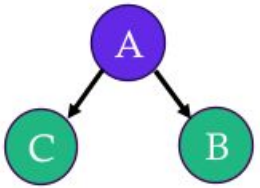
Algoritmos de Búsqueda Informada | Búsqueda Voraz (Greedy) Primero el Mejor



Cola prioritaria

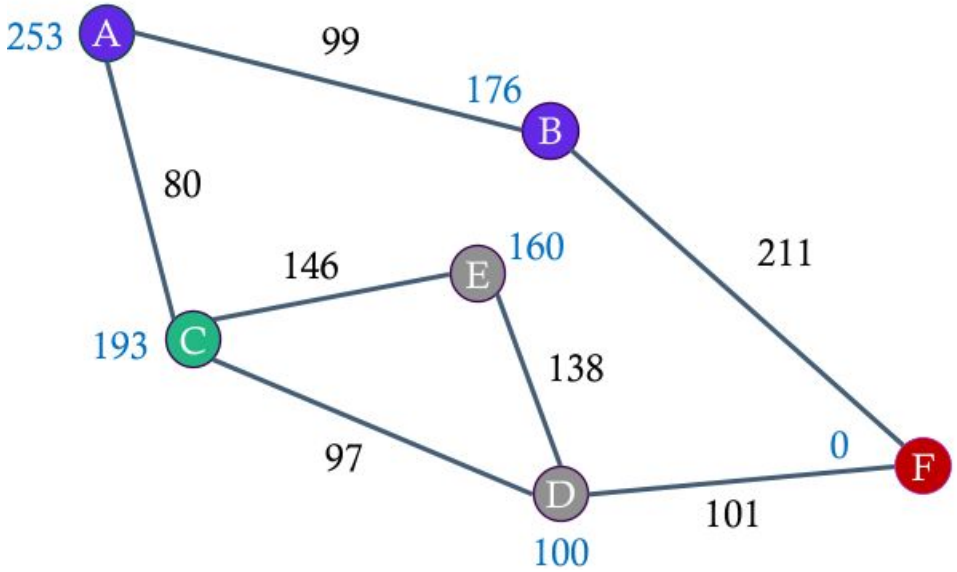
			C 193	B 176

Árbol



© gonzalezhalexis@gmail.com ©

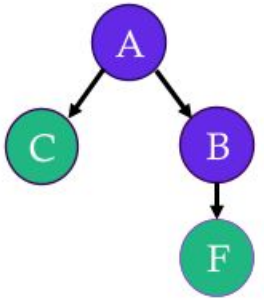
Algoritmos de Búsqueda Informada | Búsqueda Voraz (Greedy) Primero el Mejor



Cola prioritaria

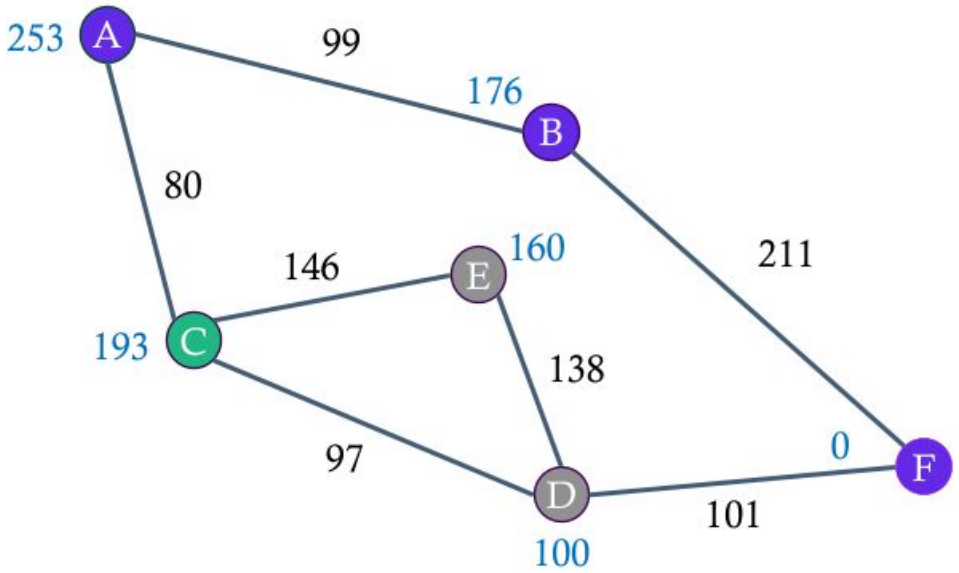
			C 193	F 0

Árbol



© gonzalezhalexis@gmail.com ©

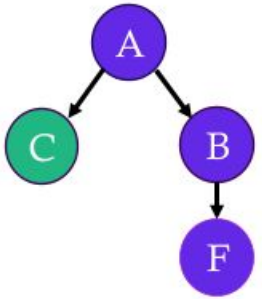
Algoritmos de Búsqueda Informada | Búsqueda Voraz (Greedy) Primero el Mejor



Cola prioritaria

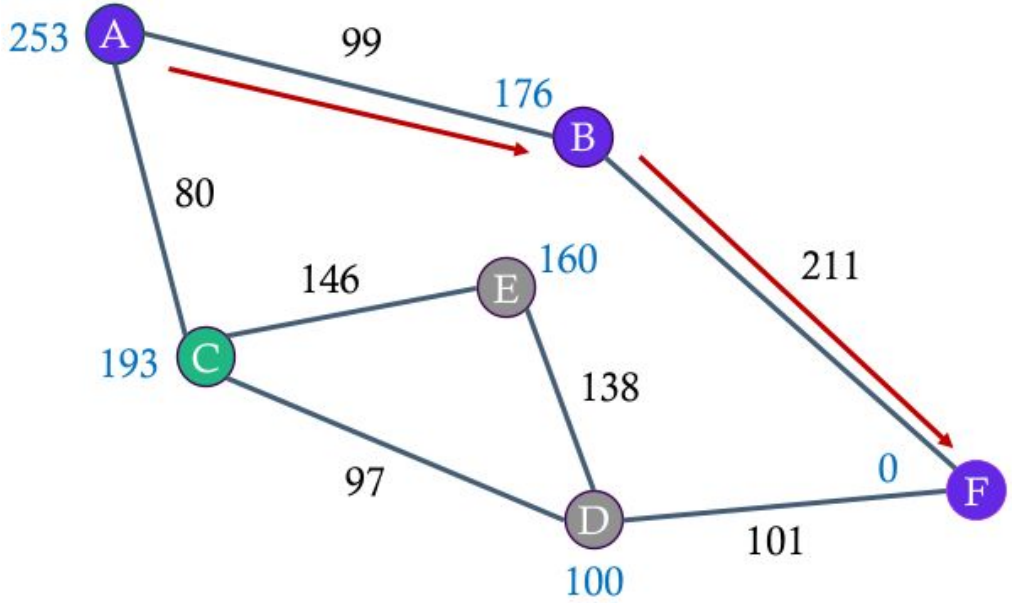


Árbol

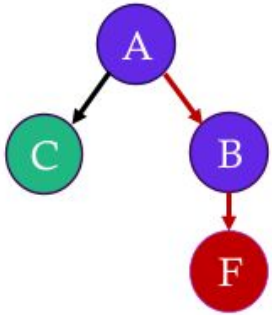


© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda Informada | Búsqueda Voraz (Greedy) Primero el Mejor



Árbol



© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda Informada | Búsqueda A*

Función de Evaluación

```
f(nodo) = costo(nodo) + h(nodo)
```

Combina el costo real acumulado con la estimación heurística al objetivo.

Admisibilidad y Compleitud

- Si todos los costos son **> 0**, se garantiza que la búsqueda es **completa**.
- Es **eficiente** si *h(nodo)* nunca sobreestima el costo real para llegar a la solución (heurística admisible).

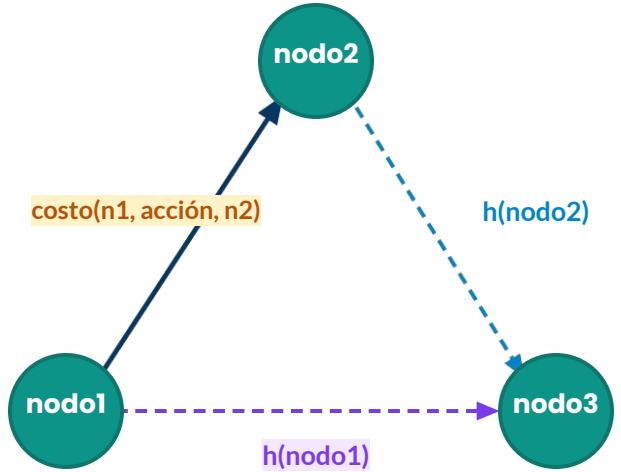
Consistencia y Optimalidad

Cumple la desigualdad triangular:

```
h(n1) < costo(n1, acción, n2) + h(n2)
```

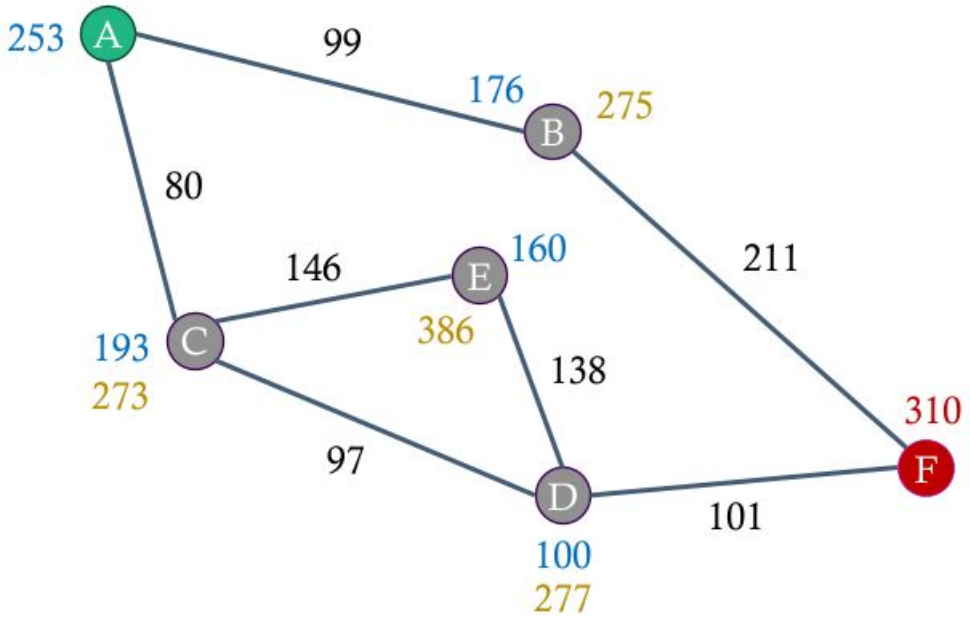
Garantía: A* siempre encuentra el camino óptimo.

Modelo de Consistencia de Nodos

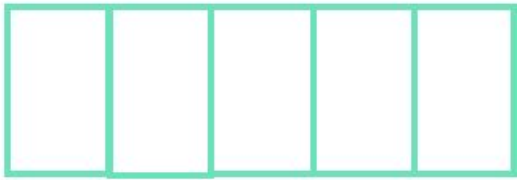


© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda Informada | Búsqueda A*



Cola prioritaria

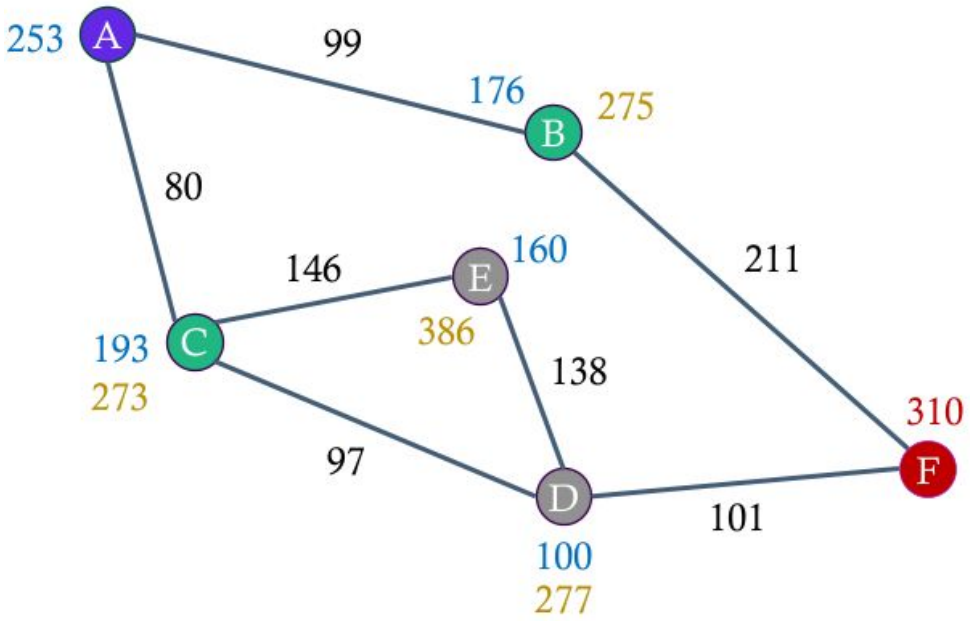


Árbol



© gonzalezhalexis@gmail.com ©

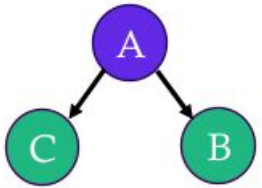
Algoritmos de Búsqueda Informada | Búsqueda A*



Cola prioritaria

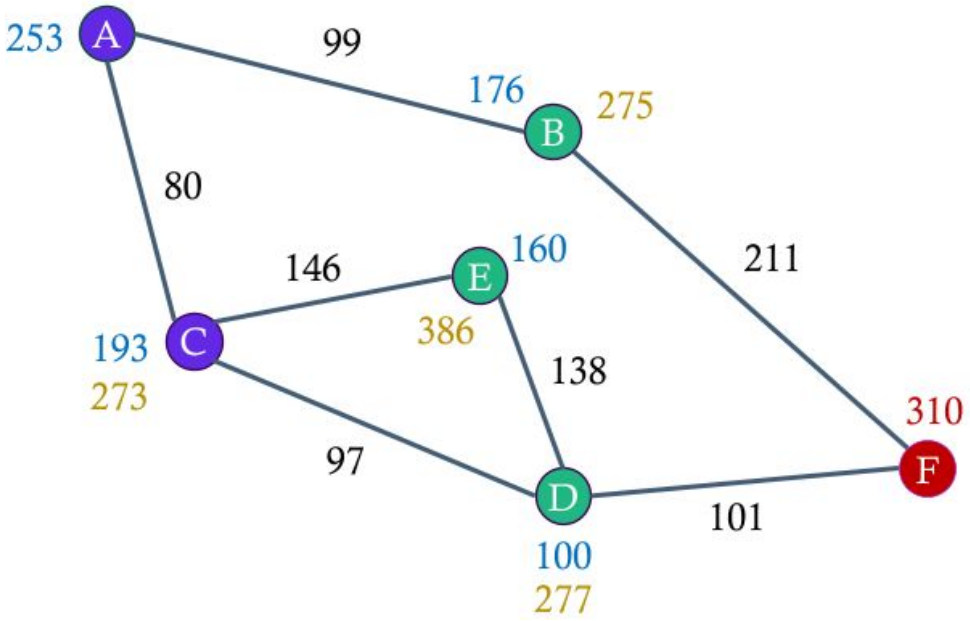


Árbol



© gonzalezhalexis@gmail.com ©

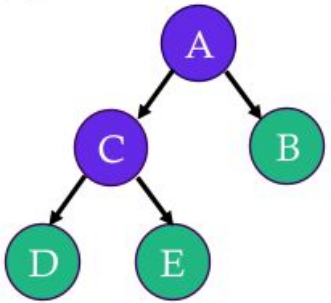
Algoritmos de Búsqueda Informada | Búsqueda A*



Cola prioritaria

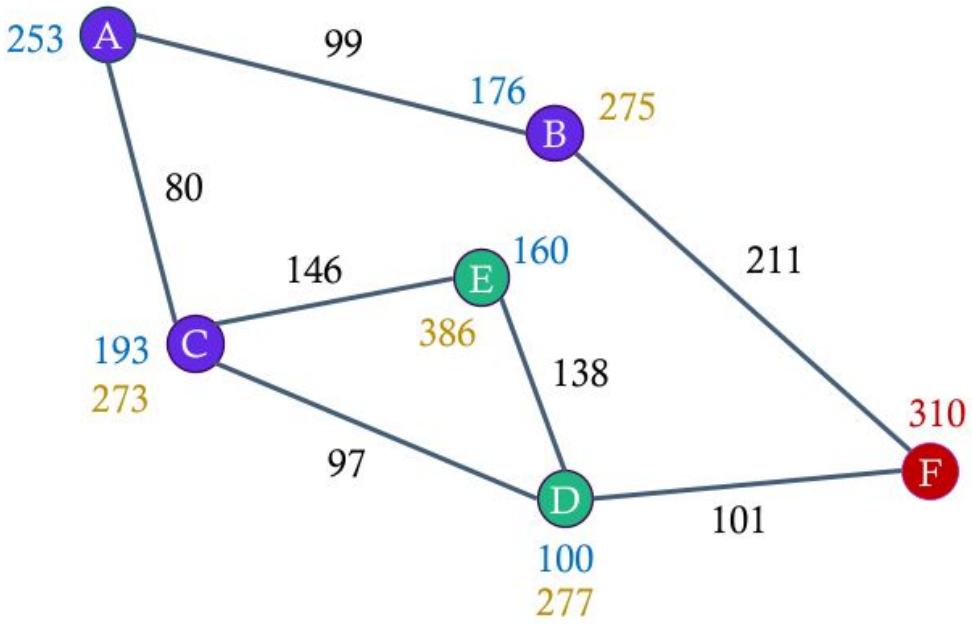
		D 277	E 386	B 275

Árbol



© gonzalezhalexis@gmail.com ©

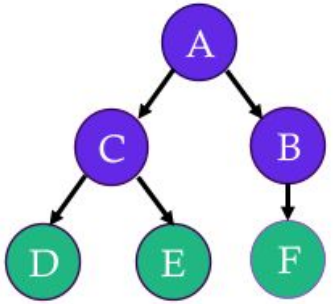
Algoritmos de Búsqueda Informada | Búsqueda A*



Cola prioritaria

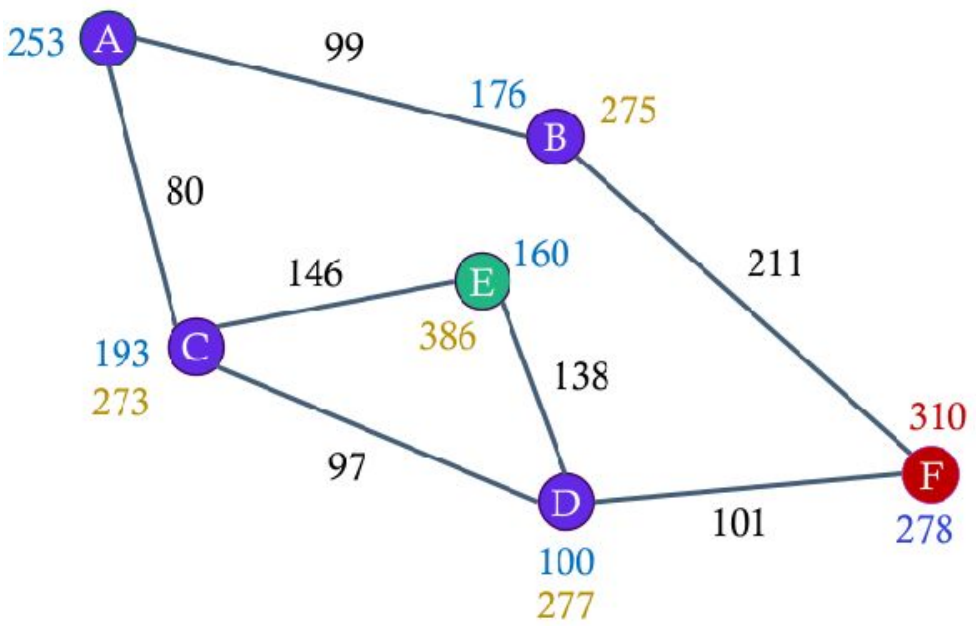
		D	E	F
		277	386	310

Árbol



© gonzalezhalexis@gmail.com ©

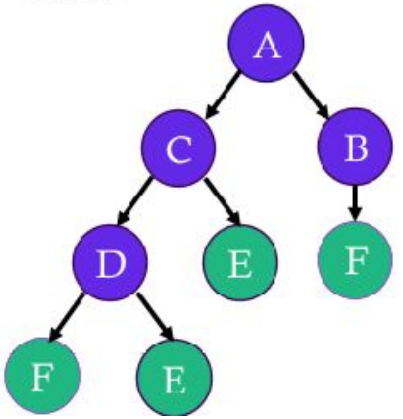
Algoritmos de Búsqueda Informada | Búsqueda A*



Cola prioritaria

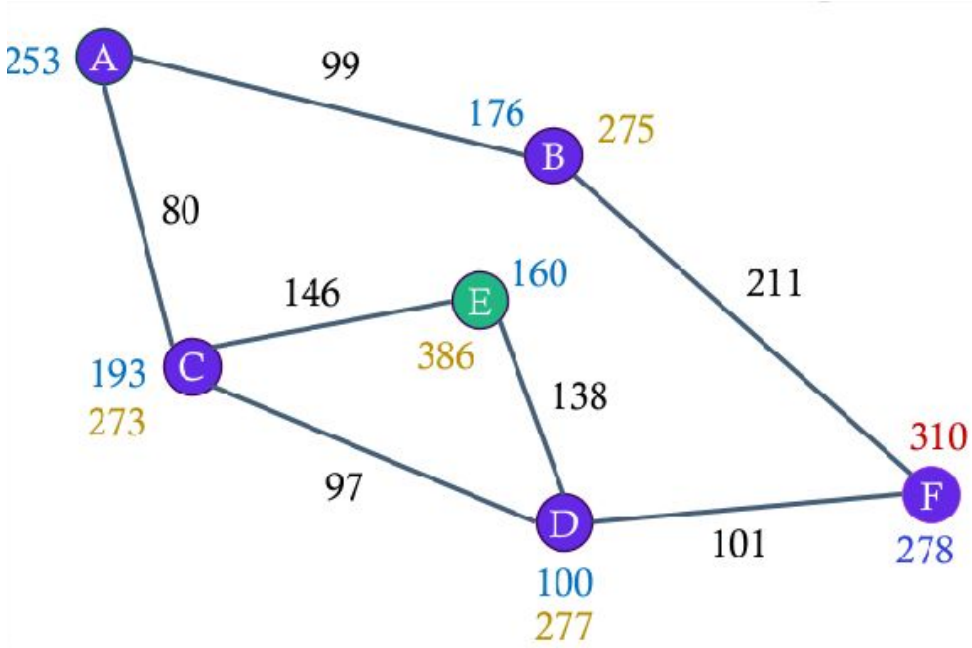
		F 278	E 386	F 310

Árbol



© gonzalezhalexis@gmail.com ©

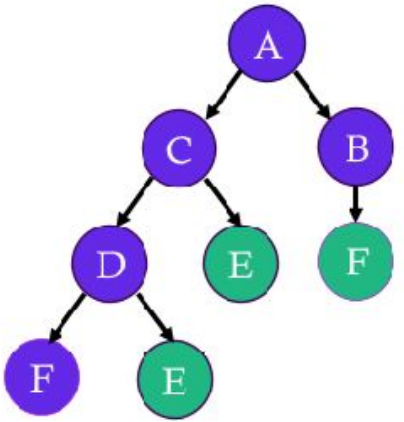
Algoritmos de Búsqueda Informada | Búsqueda A*



Cola prioritaria

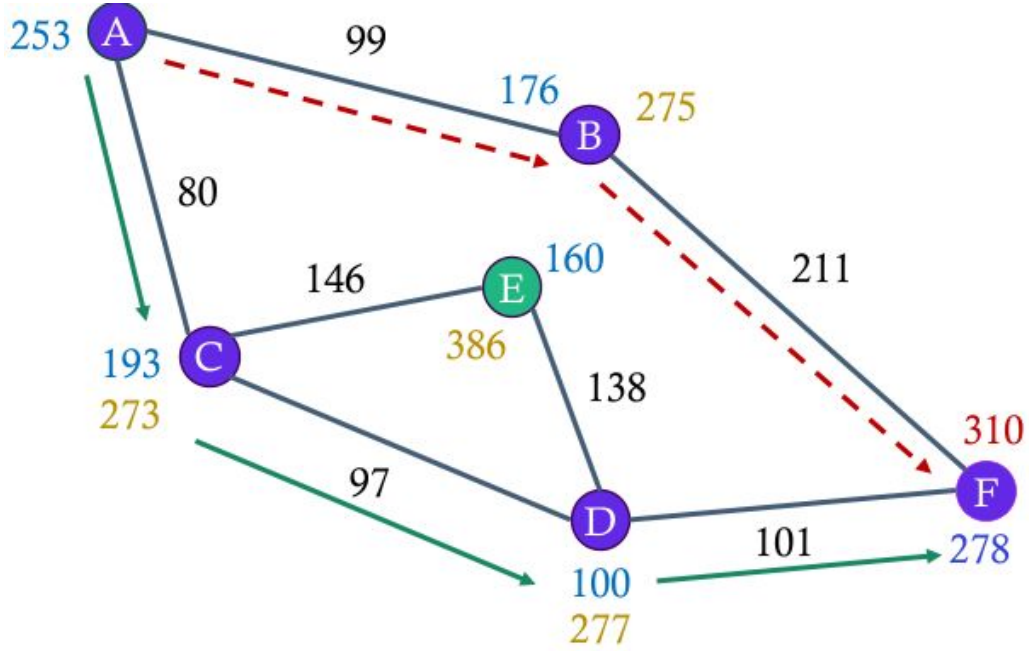
		F 278	E 386	F 310

Árbol

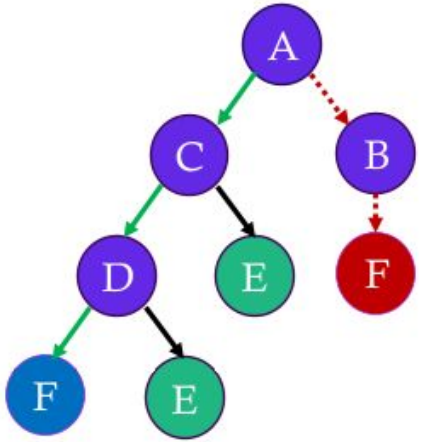


© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda Informada | Búsqueda A*

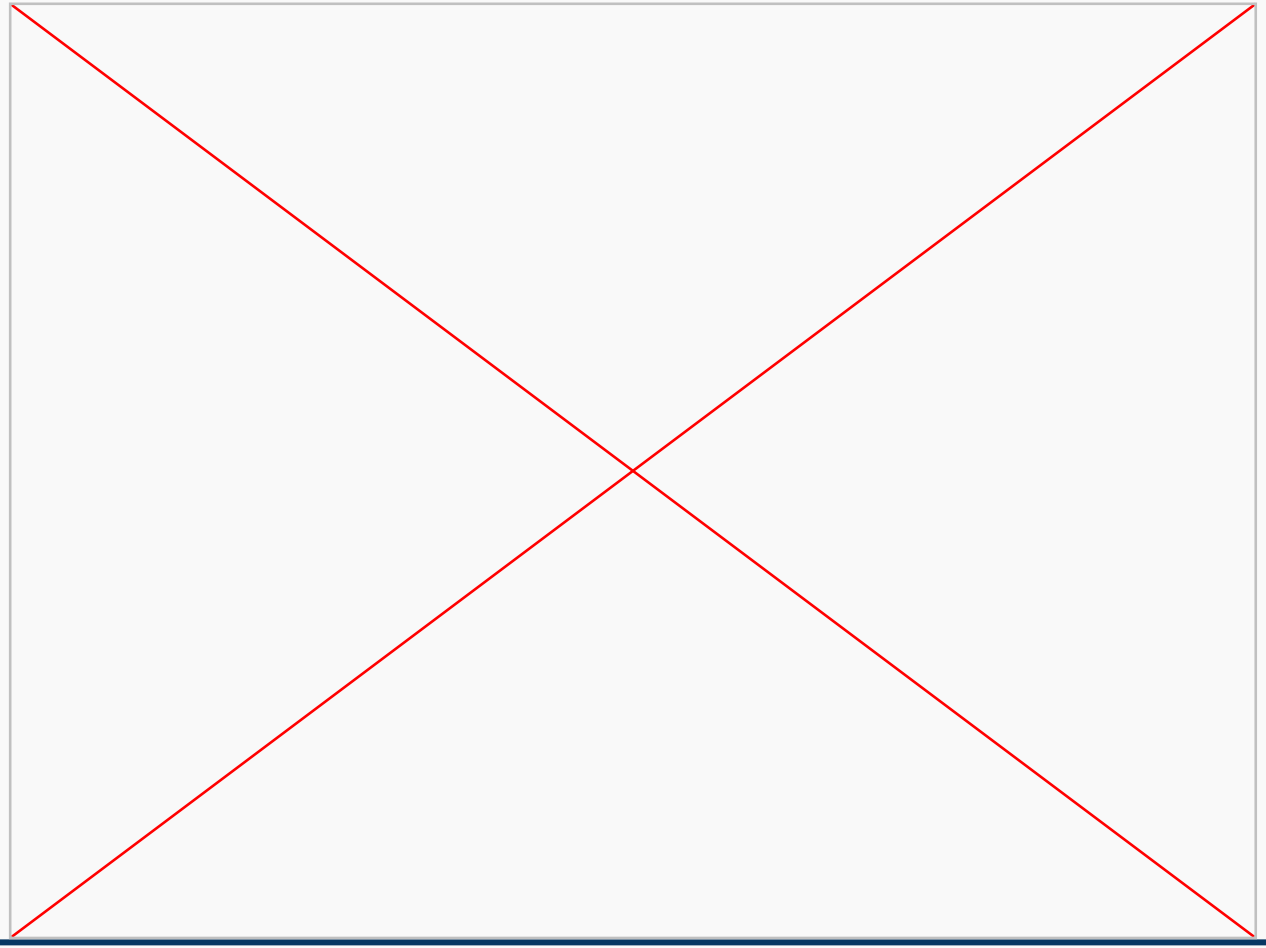


Árbol



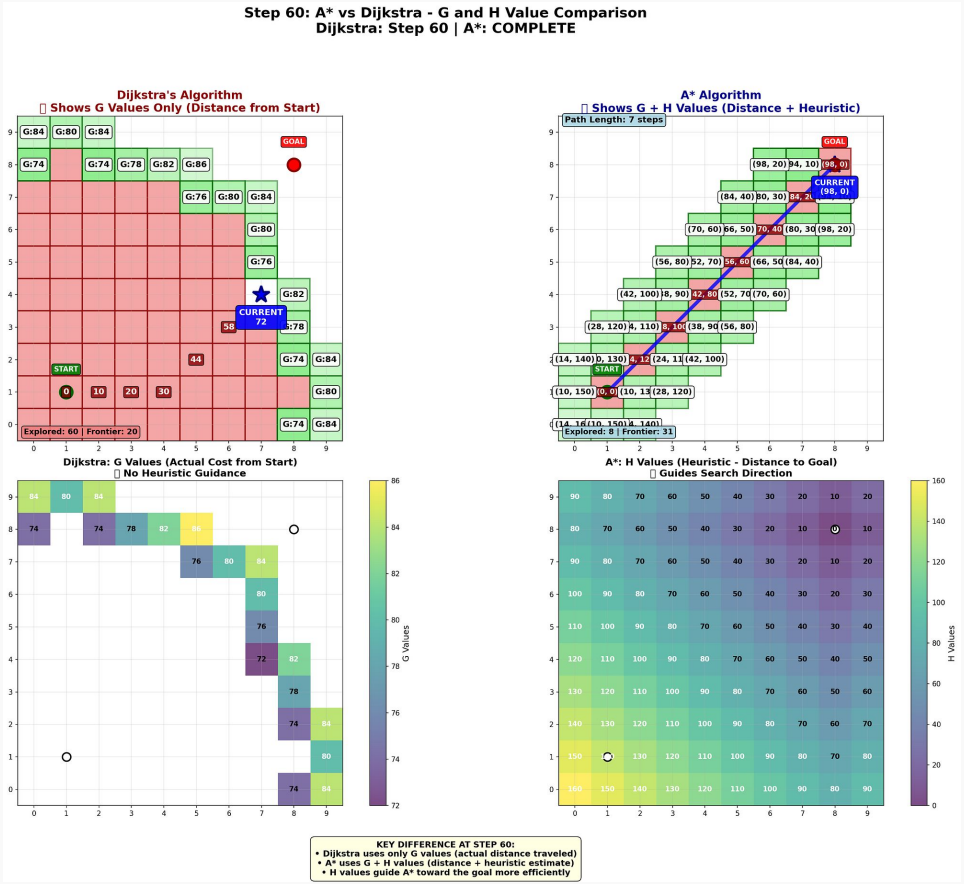
© gonzalezhalexis@gmail.com ©

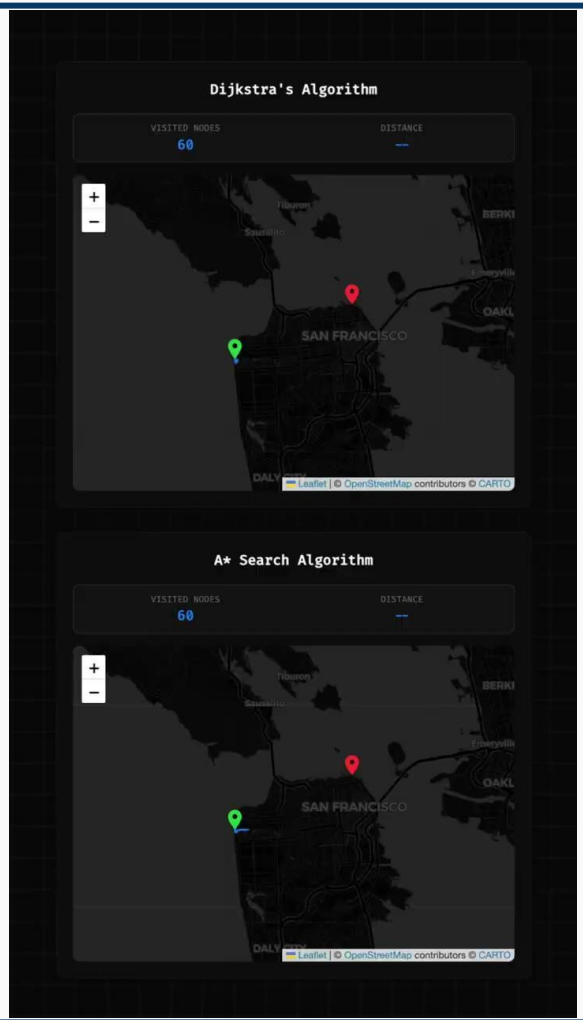
Algoritmos de Búsqueda | Comparaciones



© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda | Dijkstra vs A*





© gonzalezhalexis@gmail.com ©

Algoritmos de Búsqueda: Cuadro Comparativo

Búsqueda No Informada (Ciega)

Búsqueda Primero en Anchura (BFS)

Explora nivel por nivel usando FIFO. Garantiza solución óptima si los costos de paso son iguales.

Costo Uniforme / Dijkstra

Expande el nodo con menor costo acumulado g(n). Encuentra siempre la ruta con costo mínimo.

Búsqueda Primero en Profundidad (DFS)

Explora la rama más profunda usando LIFO. Muy eficiente en memoria, pero no garantiza optimalidad.

Profundidad Limitada (DLS)

Aplica DFS con un límite fijo de profundidad l. Evita bucles infinitos pero puede no hallar la solución.

Profundidad Iterativa (IDDFS)

Incrementa gradualmente el límite de DLS. Combina la eficiencia de memoria de DFS con la optimalidad de BFS.

Búsqueda Informada (Heurística)

Búsqueda Voraz (Greedy Best-First)

Función de evaluación: f(n) = h(n)

Expande el nodo que parece estar más cerca de la meta según la heurística h(n). Rápido, pero no garantiza encontrar la solución más corta u óptima.

ALGORITMO DESTACADO / RECOMENDADO

Búsqueda A* (A-Estrella)

Función de evaluación: f(n) = g(n) + h(n)

Combina el costo real acumulado g(n) y la estimación h(n). Es óptimo y completo si la heurística h(n) es admisible y consistente.

Criterio Clave de Selección

Sin Heurística: Usar IDDFS para memoria limitada o Dijkstra/BFS para costo óptimo sin información extra.

Con Heurística: Usar A* para optimalidad comprobada o Greedy para rapidez en grafos complejos.

© gonzalezhalexis@gmail.com ©

Guía de Decisión Arquitectónica: ¿Qué algoritmo elegir?

Criterios de selección según restricciones de hardware y requerimientos del proyecto.

Escenarios SIN Heurística (Búsqueda Ciega)

🖥️ Memoria RAM Crítica + Profundidad Desconocida

Tu elección: IDDFS (Profundidad Iterativa)

Mantiene consumo de memoria lineal (pila pequeña) pero garantiza no quedarse en un bucle infinito. Ideal para hardware limitado.

🛣️ Grafos con Costos Variables + Óptimo Absoluto

Tu elección: Dijkstra (Costo Uniforme)

Estándar de oro para la ruta más barata cuando las aristas tienen pesos diferentes (ej. redes de ruteo sin coordenadas GPS).

🌐 Grafos Simples + Aristas de Costo Igual (ej: 1)

Tu elección: BFS (Primero en Anchura)

Expande por niveles. Rápido y óptimo si todos los pasos valen lo mismo, requiriendo suficiente RAM para la cola FIFO.

Escenarios CON Heurística (Búsqueda Informada)

⭐ OPCIÓN PRINCIPAL PARA OPTIMALIDAD

Camino Óptimo Estricto (Cero margen de error)

Tu elección: A* (A-Estrella)

Condición: Requiere una heurística h(n) admisible (que nunca sobreestime el costo real).

Uso: Logística de precisión, navegación GPS y planificación robótica médica.

⚡ Velocidad Extrema + Tolerancia a Sub-óptimos

Tu elección: Búsqueda Voraz (Greedy Best-First)

Avanza velozmente hacia la meta evaluando solo la heurística h(n) e ignorando el costo pasado.

Uso: IA en videojuegos (NPCs persiguiendo objetivos) o grafos masivos.

© gonzalezhalexis@gmail.com ©

Desafío de Arquitectura: ¿Cuál es el motor adecuado?

Analicen las restricciones del sistema y elijan el algoritmo óptimo.

🤖 Caso 1: El Robot Logístico

Escenario: Un robot autónomo en un depósito masivo necesita calcular la ruta exacta para buscar un paquete. Tiene coordenadas claras del destino, no puede cometer errores porque chocaría, y el servidor central tiene RAM de sobra para guiarlo.

Solución: A* (A-Estrella)

Tenemos heurística (coordenadas GPS/grilla), necesitamos el óptimo estricto y no hay problemas de memoria.

📡 Caso 2: El Enrutador de Red

Escenario: Necesitamos rutear paquetes de datos entre servidores de distintas ciudades. Cada conexión de fibra óptica tiene una latencia (costo) diferente. No hay un "mapa geométrico" en línea recta que sirva de pista.

Solución: Dijkstra (Costo Uniforme)

Sin heurística posible, pero con costos variables y necesidad de encontrar la ruta más barata (menor latencia).

🎮 Caso 3: IA Enemiga (Videojuego)

Escenario: Los NPCs de un juego de disparos deben perseguir al jugador que se mueve rápido. El cálculo debe hacerse a 60 fps, por lo que debe ser casi instantáneo. Si el NPC toma una curva ligeramente más larga, el jugador ni lo nota.

Solución: Búsqueda Voraz (Greedy)

Prima la velocidad extrema; tenemos la posición del jugador (heurística), pero toleramos una ruta sub-óptima.

💻 Caso 4: El Microcontrolador

Escenario: Un firmware en un ESP32 debe explorar combinaciones de un sistema de seguridad para encontrar la clave. El espacio de combinaciones es gigantesco y la placa tiene apenas unos pocos kilobytes de RAM libre.

Solución: IDDFS (Profundidad Iterativa)

Sin información del objetivo (ciega) y con memoria crítica, necesitamos la eficiencia LIFO sin quedar atrapados.

👥 Caso 5: La Red de Contactos

Escenario: Encontrar la menor cantidad de "saltos" (conexiones mutuas) entre dos usuarios en una red social. Todos los saltos valen exactamente 1 y el servidor tiene amplia capacidad de memoria.

Solución: BFS (Primero en Anchura)

Costos idénticos, por lo que buscar capa por capa nos garantiza la cadena de amigos más corta rápidamente.

© gonzalezhalexis@gmail.com ©

Google Colab

Ejemplo en Python

07_agentes_y_busqueda



© gonzalezhalexis@gmail.com ©

Actividad individual opcional





CONSIGNA

Implementen un algoritmo de búsqueda (excepto BFS) para resolver la Torre de Hanoi. La nota máxima varía según el algoritmo:

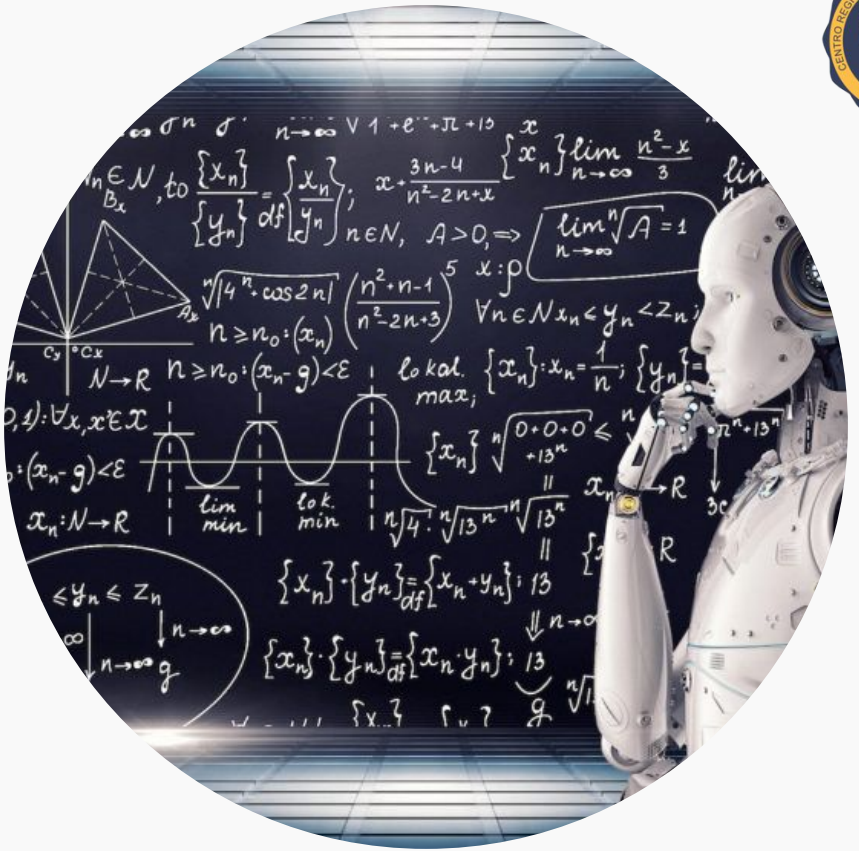
- Profundidad / Costo Uniforme: nota máx. 6.
- Profundidad Iterativa: nota máx. 7.
- Voraz (heurística aula): nota máx. 8.
- Voraz (heurística propia) / A* (heurística aula): nota máx. 9.
- A* (heurística propia): nota máx. 10.

Entrega	Preguntas correctas	Puntos
No	-	0
Si	Menor a 8	0
Si	Mayor o igual a 8	1

© gonzalezhalexis@gmail.com ©

¡Gracias!

¿Consultas?



Ing. González H. Alexis

© gonzalezhalexis@gmail.com ©